分布式机器学习

实验指导书

课程号：85990072

授课教师：王智副教授

编者：袁新杰助教（23 春）
代诗琦助教（24 春）
解书照助教（24 秋）
李博文助教（25 秋）
李博文助教（26 秋）

清华大学
清华大学深圳国际研究生院



清華大學深圳国际研究生院
Tsinghua Shenzhen International Graduate School

[最后编辑于 2026 年 10 月 25 日]

前言-26 秋季学期

本学期实验指导书在上一学期版本的基础上修订而成，主要修改概括如下：

1. 根据去年的教学反馈以及今年人工智能领域的发展，对实验内容和要求进行了调整，更加注重对实验过程与结果的分析，而非单纯的代码编写；
2. 新增了部分思考题与实验注意事项，帮助同学们更好地把握实验重点；
3. 修正了一些勘误。

感谢授课老师以及历届助教同学对本课程的贡献。本学期实验指导书在线资源通过如下地址发布：https://github.com/realBowenLi/dml_exp_26fall。

助教李博文 于 2026 年 10 月

前言-25 秋季学期

本学期的实验指导书在现有基础之上丰富了实验内容，并且平衡了不同实验的任务量。相比上一版本，本学期实验指导书的主要修改如下：

1. 由于院系算力资助情况的变化，对指导书有关 MindSpore 框架的使用教程位置进行了调整，并且对同学们完成实验使用的机器学习框架不做硬性要求；
2. 为实验一增加了 pytorch profiler + tensorboard 的内容，有助于同学理解分布式机器学习过程中的底层运算逻辑。为实验二增加了有关梯度压缩的思考题；
3. 修正了一些勘误。

感谢授课老师以及历届助教同学对本课程的贡献。本学期实验指导书在线资源通过如下地址发布：https://github.com/realBowenLi/dml_exp_25fall。

助教李博文 于 2025 年 10 月

前言-24 秋季学期

本学期实验指导书请见：https://github.com/ShuzhaoXie/dml_exp_24fall。相比上一学期：

1. 增加部分思考题；
2. 删去学校资源的用法；

助教解书照 于 2024 年 10 月

前言-24 春季学期

本学期实验指导书保留上一年丰富详实的内容外，进一步完善课程实验设置、平衡各项实验难度。修订版实验指导书通过如下 github 地址写作和发布：https://github.com/shiqi-dai/Distributed-Machine-Learning-Experiment-Document-24-Spring

实验内容所做修改如下：

由于同学对学习使用框架 Mindspore 的积极性不高的问题，我们在难度最低的实验一：梯度下降单机优化中要求同时采用 Pytorch 和 MindSpore 框架写优化器类，旨在调动大家查阅说明文档学习新框架的能力。同时将实现的优化器类改为 SGDM,Adam 两个在 CV、NLP、RL、语音合成等领域的优化方法，贴近平日科研任务所用。

对于实验报告撰写存在的各式各样的问题，我们在实验指导书新增一节实验报告撰写要求，并且明确在实验三：数据并行的实验报告要求中增加报告内容，确保同学们更准确明白掌握实验内容。

本课程热情欢迎各位同学共同构建课程知识库，可以将自己对实验指导书的建议另附在实验报告中，您对实验指导书的贡献将会被酌情考虑，可能会影响最终的评分。

助教代诗琦 于 2024 年 2 月

前言-23 春季学期

本指导书初版写于 2022–2023 学年春季学期初。在该学期以前，本课程也并非没有实验指导书，只是之前的都是幻灯片形式的，而我为什么想要把它一本书呢？这有两点原因：

首先，我要确保我理解了这些实验内容，这样我才能较为自信的为同学们讲解，避免以其昏昏使人昭昭。实际上，这对我而言并不简单，我此前并没有上过这门课程，对机器学习的了解也较为浅薄。为了能当好助教，我要先自学这门课程，至少要先把这些实验内容实验搞明白。然而，以往的幻灯片形式的指导书以图片为主，语言为辅，当缺乏必要的讲解时，以它为主要资料会让基础较为薄弱的我学习起来感到十分痛苦。为此，一方面为了证明我已经掌握了这些实验，另一方面为了让像我一样的缺乏基础的同学更容易理解，我决定用自己的方式重新写一遍这本指导书。

其次，由于一个人的能力是有限的，我希望该指导书可以由历届助教以及所有上课的同学们一起参与完成，使得该指导书能够更好的传承、历久弥新。为了让更多的人参与到指导书的编写中来，该指导书的形式或许相较于幻灯片形式更为合适。一方面，使用 latex 语言，大家可以方便地在 github 或 gitee 中写作；另一方面，书本相比幻灯片更成体系，大家可以更加方便地在目录中为自己想要补充的内容找到合适位置。因此，我也在此呼吁选课的各位同学来分享自己的知识。

该指导书第一章介绍环境配置，包括本地环境、本地虚拟环境的配置以及华为云、深研院计算资源的使用。第二章中简要介绍了华为 MindSpore 框架及其安装。第三至六章介绍四次实验内容，以及必要的 pytorch 中的函数、方法，以及模拟多节点的方法；其中在实验一中，我们还介绍了使用 MindSpore 完成实验的方法。

本实验指导书编写过程中也收到了去年的助教王晓禅同学，以及王智老师实验室内选过该课程的代诗琦、吴鸣洲同学的帮助，在此向他们表示感谢。

该实验指导书通过如下 github 地址协作和发布：https://github.com/cantjie/Distributed-Machine-Learning-Experiment-Document

助教袁新杰　于 2023 年 3 月

目录

前言 . . . . . .
目录 iv

1 环境配置与计算资源使用 1
1.1 使用本地环境与 GPU 1
1.1.1 安装 CUDA 工具箱 2
1.1.2 安装 Anaconda 5
1.1.3 创建虚拟环境并安装 PyTorch 5
1.1.4 安装其他包 7
1.2 使用虚拟环境与本地 GPU 7
1.2.1 安装并配置 Docker 引擎 7
1.2.2 搜索并下载 PyTorch 镜像 9
1.2.3 启动容器 11
1.2.4 安装新包后重新打包成镜像 11
1.2.5 限制 Docker 内存占用 (可选) 13

2 实验一：梯度下降单机优化 14
2.1 实验内容与要点介绍 14
2.1.1 实验内容与要求 14
2.1.2 PyTorch 优化器 15
2.1.3 MindSpore 优化器 16
2.1.4 背景知识简单回顾 17
2.2 使用可视化工具观察训练期间显存与时间占用情况 18
2.3 使用 VSCode 与本地环境调试运行 20
2.4 使用 VSCode 与本地容器调试运行 21
2.4.1 启动容器并挂载本地文件夹 21
2.4.2 在 VSCode 中使用容器 22
2.5 使用 VSCode 与远程服务器调试运行 22
2.5.1 使用远程服务器计算资源 22
2.5.2 使用华为云计算资源 24
2.5.3 使用密钥对登录远程服务器 (可选) 25

3 实验二：通信模型与参数聚合 29
3.1 实验内容与要点介绍 29
3.1.1 实验内容与要求 29
3.1.2 多节点通信 30
3.1.3 记录 GPU 上任务的运行时间 31

3.2 使用进程模拟多节点 . . . 32
3.2.1 手动运行多进程 . . . 32
3.2.2 使用 torch.multiprocessing 自动创建多进程 . . . 32
3.3 使用容器模拟多节点 . . . 33
3.3.1 Docker compose 介绍 . . . 33
3.3.2 通过 Docker compose 启动容器 . . . 34

4 实验三 (1)：数据并行 36
4.1 实验内容与要点介绍 . . . 36
4.1.1 实验内容与要求 . . . 36
4.1.2 数据集、加载器和采样器 . . . 37

5 实验三 (2)：模型并行实验 39
5.1 实验内容与要点介绍 . . . 39
5.1.1 实验内容与要求 . . . 39
5.1.2 RPC 框架介绍 . . . 40

6 实验报告撰写要求 43

7 实验验收列表:2026 秋季学期 45
7.1 实验一：单机优化算法构建 . . . 45
7.2 实验二：通信模型与参数聚合 . . . 45
7.3 实验三：数据并行和模型并行 . . . 45

8 华为云资源与 MindSpore 框架使用方法 47
8.1 使用华为云计算资源 . . . 47
8.1.1 创建开发环境 . . . 47
8.1.2 添加数据存储 . . . 50
8.2 MindSpore 介绍 . . . 51
8.2.1 整体介绍 . . . 51
8.2.2 MindSpore 安装 . . . 53
8.2.3 社区资源 . . . 53

第 1 章　环境配置与计算资源使用

本章将主要介绍使用本地计算资源构建环境的方法。同学们在进行实验时，原则上使用本章任意一节的知识即可完成所有实验，但推荐同学们掌握多种方法。相信大多数同学在课前已经掌握了 §1.1 节中本地环境的配置方法，而 §1.2 节中介绍的 Docker 的使用则不为大多数同学所熟知，因此推荐同学掌握 Docker 的使用方法，Docker 在今后的科研和开发中也有望帮助同学们事半功倍。

1.1　使用本地环境与 GPU

使用本地计算资源可以不收网络链接状况约束，随时随地调试程序，对于简单的项目，本地调试也可能更省时间。

本节以助教所使用的计算机为例，展示环境配置过程。助教使用的计算机系统与配置为：

- 系统：windows10 专业教育版；22H2
- 处理器：Intel(R) Core(TM) i7-8700 CPU
- 内存：16GB
- 显卡：NVIDIA GeForce RTX 2060
- 编辑器：Visual Studio Code

1.1.1 安装 CUDA 工具箱

对于包含英伟达显卡的计算机，我们推荐首先安装 CUDA 工具包以使用 GPU 加速计算。

注意，仅包含英伟达 GPU 的计算机需要安装 CUDA 工具箱以使用 GPU 加速计算。使用核显或 AMD 显卡的计算机再后续步骤中使用 CPU 计算即可。

GPU 型号、CUDA 工具包、PyTorch 版本相互关联。因此需要一起规划好。

首先查看自己的显卡的算力 https://developer.nvidia.com/zh-cn/cuda-gpus：查看得到我的显卡的 2060 的算力为 7.5, 图 1.1。



Figure 1.1: fig:nvidia-rtx-2060-capability

然后根据算力查看支持该算力的 CUDA 版本 https://en.wikipedia.org/wiki/CUDA：查看得到支持我显卡的 CUDA 版本为 ≥10.0，图 1.2。

同时 CUDA 对显卡驱动的最低版本也提出了要求 https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/index.html，但显卡驱动对 CUDA 向下兼容，因此一般安装了最近发布的显卡驱动版本即可，无需与 CUDA 版本特别对应。

GPUs supported [edit]

Supported CUDA Compute Capability versions for CUDA SDK version and Microarchitecture (by code name):

Compute Capability (CUDA SDK support vs. Microarchitecture)

CUDA SDK version(s)	Tesla	Fermi	Keppler (early)	Kepler (late)	Maxwell	Pascal	Volta	Turing	Ampere	Ada Lovelace	Hopper
1.0[29]	1.0 – 1.1										
1.1	1.0 – 1.1+x										
2.0	1.0 – 1.1+x										
2.1 - 2.3.1[30][31][32][33]	1.0 – 1.3										
3.0 - 3.1[34][35]	1.0 –	2.0									
3.2[36]	1.0 –	2.1									
4.0 - 4.2	1.0 –	2.1+x									
5.0 - 5.5	1.0 –			3.5							
6.0	1.0 –			3.5							
6.5	1.1 –				5.x						
7.0 - 7.5		2.0 –			5.x						
8.0		2.0 –				6.x					
9.0 - 9.2			3.0 –				7.0				
10.0 - 10.2			3.0 –					7.5			
11.0[37]				3.5 –					8.0		
11.1 - 11.4[38]				3.5 –					8.6		
11.5 - 11.7.1[39]				3.5 –					8.7		
11.8[40]				3.5 –							9.0
12.0					5.0 –						9.0

Figure 1.2: fig:corresponding-cuda-version

最后要注意，PyTorch 并不一定支持最新的 CUDA 版本，因此安装前再去 PyTorch 上看一眼 PyTorch 支持哪些 CUDA 版本，图 1.3：https://pytorch.org/get-started/locally/

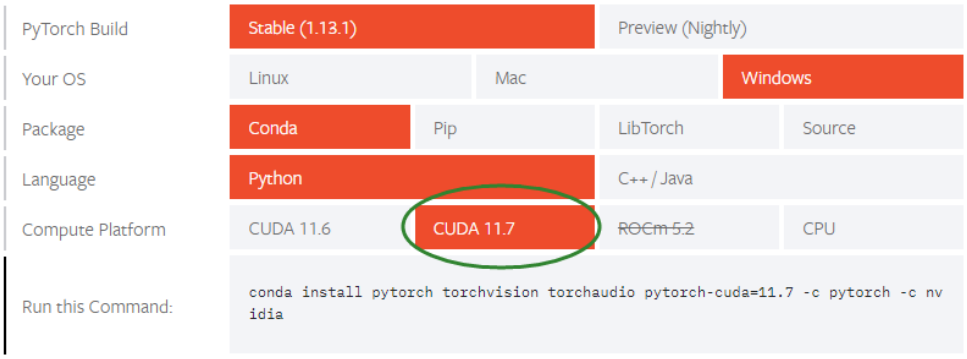


Figure 1.3: fig:cuda-version-constrained-by-pytorch

我们发现 PyTorch 最高支持到 CUDA 11.7，满足显卡算力对 CUDA 版本 ≥10.0 的要求，因此我们可以选择安装 CUDA 11.7。

选择对应版本 CUDA 安装包并下载安装，安装过程略，图 1.4：https://developer.nvidia.com/cuda-toolkit-archive

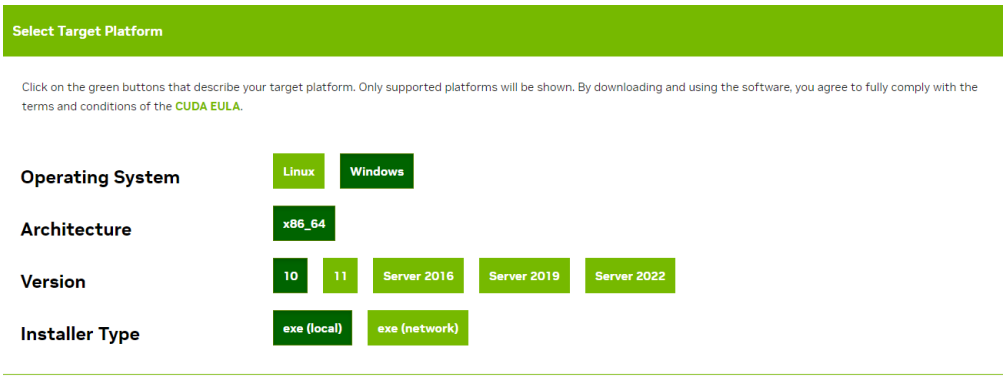


Figure 1.4: fig:select-cuda-version

在这一步完成后，我们打开终端输入 `nvcc -V` 以及 `nvidia-smi` 应当分别能看到图 1.5 和图 1.6 类似的输出，这说明我们安装完成。

```
(base) PS C:\Users\MMLab_Cantjie> nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Tue_Mar__8_18:36:24_Pacific_Standard_Time_2022
Cuda compilation tools, release 11.6, V11.6.124
Build cuda_11.6.r11.6/compiler.31057947_0
```

Figure 1.5: caption:nvcc-v-install-success

```
(base) PS C:\Users\MMLab_Cantjie> nvidia-smi
Sun Mar  5 11:03:16 2023
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 531.18                 Driver Version: 531.18       CUDA Version: 12.1     |
|-------------------------------+----------------------+----------------------+
| GPU  Name                     TCC/WDDM | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf            Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|===============================+======================+======================|
|   0  NVIDIA GeForce RTX 2060       WDDM | 00000000:01:00.0  On |                  N/A |
| 45%   36C    P8               13W / 160W|   1071MiB /  6144MiB |      5%      Default |
|                                         |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|    0   N/A  N/A      6964    C+G   C:\Windows\explorer.exe                 N/A      |
```

Figure 1.6: caption:nvidia-smi-install-success

1.1.2 安装 Anaconda

我们可能同时有多个项目或作业在处理，而不同的项目或作业可能使用了不同 python 版本、不同的工具包等，为了避免冲突，我们通常会为每一个项目或作业指定一个虚拟环境，以使得各个环境之间互不干扰。为此，我们 Anaconda 以创建并管理虚拟环境。

安装过程参考官网文档即可：https://docs.anaconda.com/anaconda/install/windows/

安装完成后启动终端，输入 `conda-V`，如正确显示 conda 版本则说明安装成功。

```
(base) PS C:\Users\MMLab_Cantjie> conda -V
conda 22.9.0
```

Figure 1.7: caption:conda-install-success

1.1.3 创建虚拟环境并安装 PyTorch

安装完成 conda 后，我们新建一个预装了 Python 的、用来完成本门课程的虚拟环境。

需要注意的是，PyTorch 和 Python 版本也需要对应，在 https://github.com/pytorch/vision#installation 中，我们发现 torch 1.13 要求 python 介于 3.7.2 和 3.10 之间。

torch	torchvision	python
main / nightly	main / nightly	>=3.8, <=3.10
1.13.0	0.14.0	>=3.7.2, <=3.10
1.12.0	0.13.0	>=3.7, <=3.10
1.11.0	0.12.0	>=3.7, <=3.10
1.10.2	0.11.3	>=3.6, <=3.9
1.10.1	0.11.2	>=3.6, <=3.9

Figure 1.8: caption:pytorch-python-version-compability

打开终端，输入下面命令以利用 conda 新建环境，

```
$ conda create --name <envname> python=3.9
```

将其中 <envname> 改成自定义的环境名称，如助教自己选择的 distributedml。

新建完成后，通过 `condaactivate<envname>` 进入环境。在 pytorch 官网安装页面 https://pytorch.org/get-started/locally/ 选择对应的 pytorch 版本、系统版本等，复制给出的命令并运行。

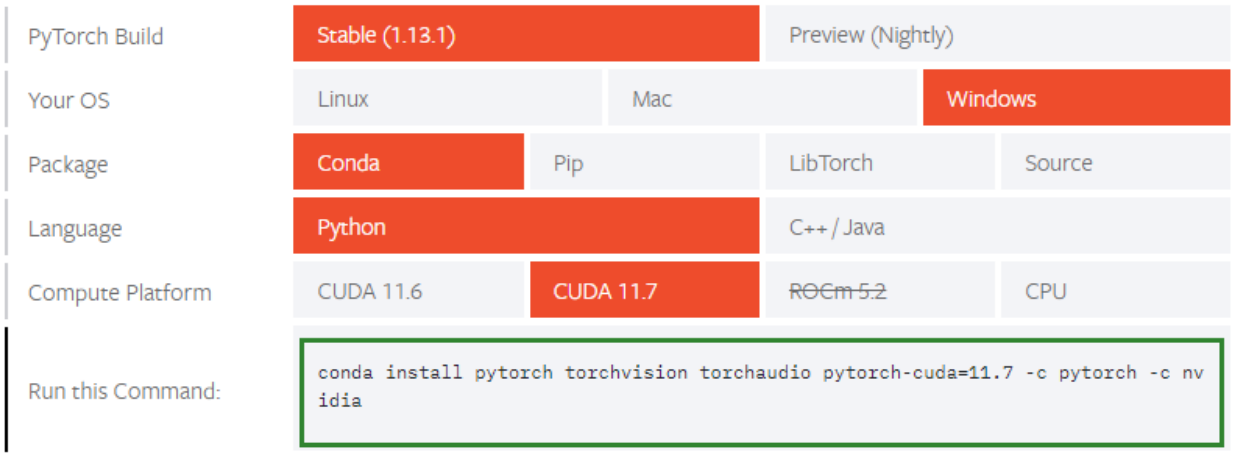


Figure 1.9: caption:pytorch-install-command

安装完成后，进入 Python 就可以 `importtorch` 了，如图 1.10.

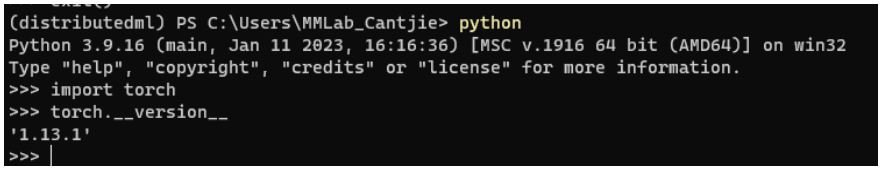


Figure 1.10: caption:pytorch-install-success

1.1.4 安装其他包

如果我们还想要安装其他包，比如同学们画图常用的 matplotlib 包，该怎么办呢？在 `condaactivate<envname>` 进入环境后，直接通过 `condainstallmatplotlib` 就可以了。

1.2 使用虚拟环境与本地 GPU

上面的本地环境配置不可为不复杂，CUDA、显卡型号、显卡驱动、PyTorch、Python 等版本需要手动一一对应起来安装。那有没有什么更简单的利用本机 GPU 计算资源的方法呢？

在这一节，我们介绍直接利用 Docker 镜像搭配环境的方法。

1.2.1 安装并配置 Docker 引擎

首先在官网下载安装包 https://docs.docker.com/desktop/install/windows-install/，安装过程略。

在安装完成后启动 Docker Desktop，在 windows 下，很可能会报错（具体内容是啥助教忘了截图了），一般错误的原因是缺少 wsl2 和 hyper-v。

为了启用 hyper-v，在控制面板中按照图 1.11 中的操作选中 Hyper-V 并确定。

为了启用 wsl2，参考 https://learn.microsoft.com/en-us/windows/wsl/install，在终端下输入 `wsl--install` 等待安装完成即可。

安装完成后启动 Docker Desktop，为了加速下载，可以按照图 1.12 所示方法为 Docker 指定国内镜像服务器，即在原本的配置中加入如下内容。

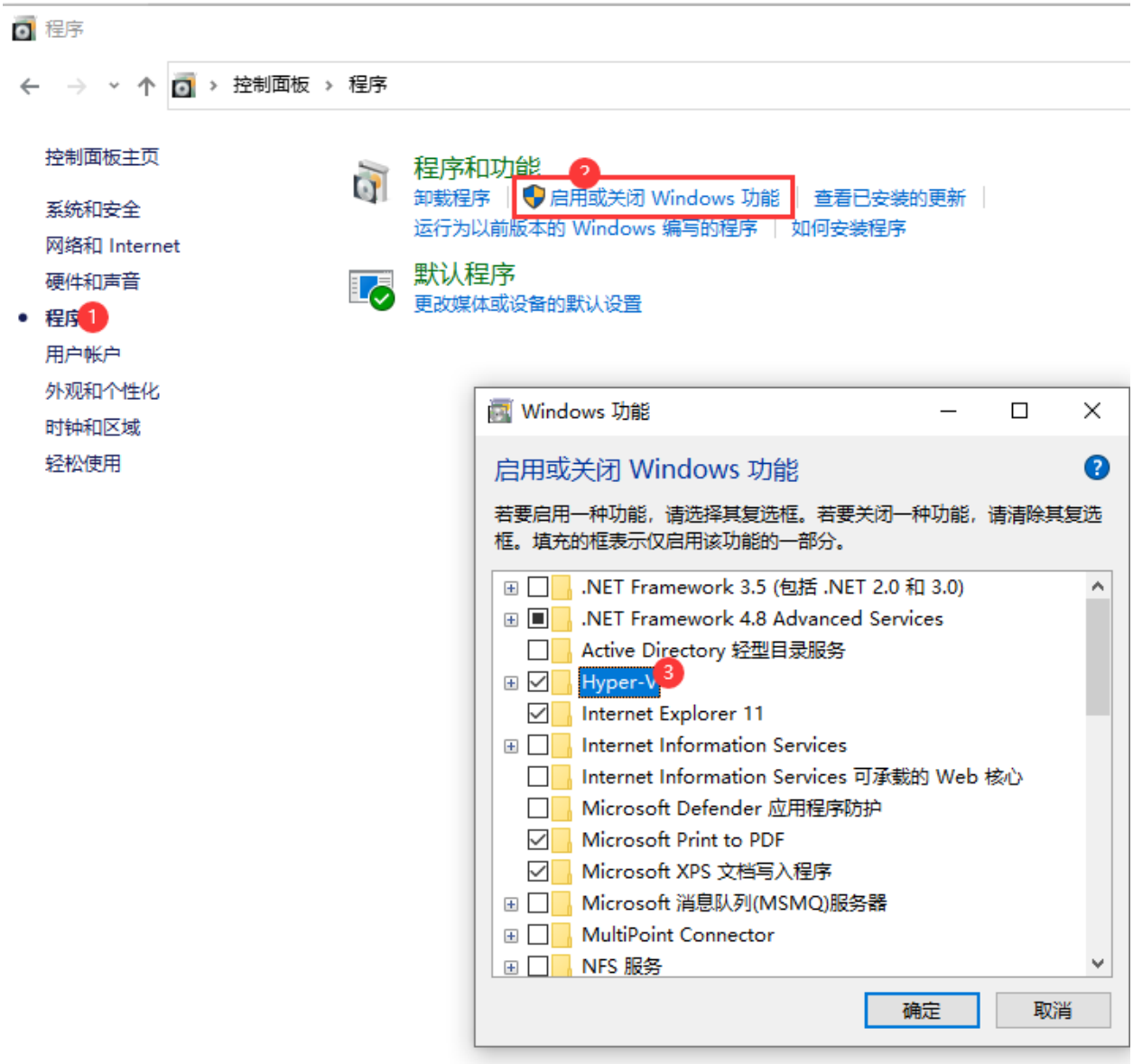


Figure 1.11: caption:turn-on-hyper-v

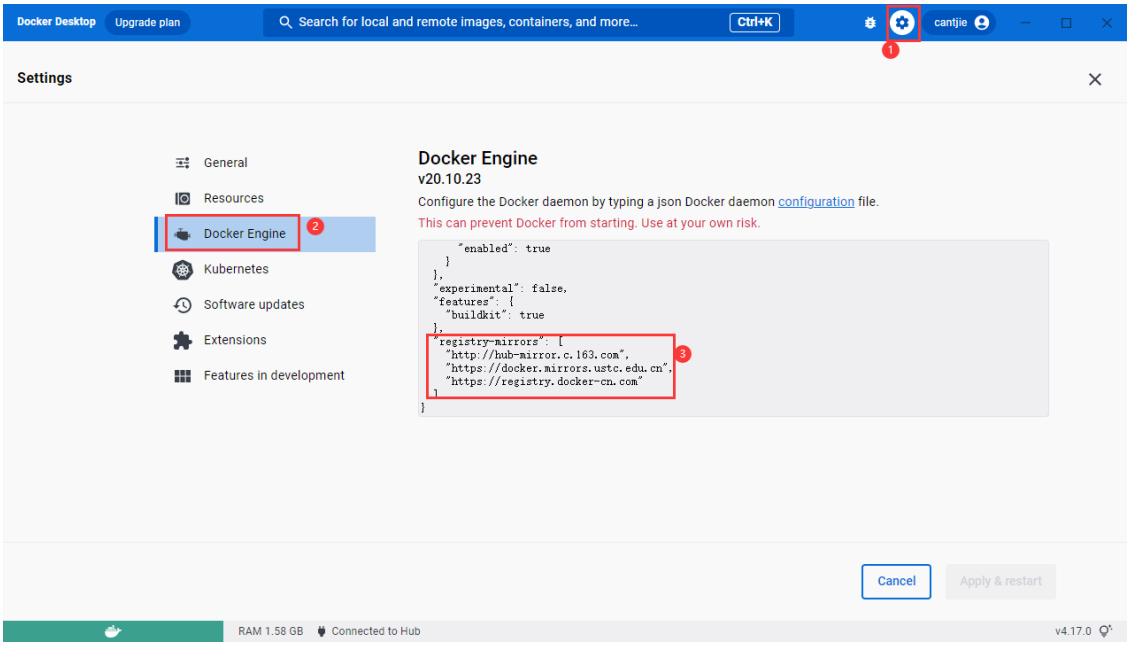


Figure 1.12: caption:docker-mirrors-setting

```
    "registry-mirrors": [
        "http://hub-mirror.c.163.com",
        "https://docker.mirrors.ustc.edu.cn",
        "https://registry.docker-cn.com"
    ]
```

启动终端，输入 `docker--version`，如图 1.13，正常返回 Docker 版本就说明安装成功了。

```
(distributedml) PS C:\Users\MMLab_Cantjie> docker --version
Docker version 20.10.23, build 7155243
```

Figure 1.13: caption:docker-install-success

1.2.2 搜索并下载 PyTorch 镜像

Dockerhub 是一个共享镜像的平台 https://hub.docker.com/。所谓镜像，类似于一个操作系统的 iso 文件：我们拿到 iso 文件后可以创建使用该操作系统的虚拟机；而当我们拿到镜像后，也可以利用该镜像创造一个使用该镜像的容器，即容器是一个镜像的实例。

因此，如果有人在某个容器中把 CUDA、PyTorch、Python 等环境都配置好，并打包成镜像共享给我们，我们就可以免去复杂的安装过程，从而直接使用镜像生成容器，在容

器中直接运行我们所写的脚本。

在 DockerHub 中，我们搜索 `pytorch/pytorch`，可以找到对应的这个镜像 https://hub.docker.com/r/pytorch/pytorch。点击网页中的 Tags 标签页，我们可以从图 1.14 看到这个镜像就是已经把 PyTorch 和 CUDA 安装好了的，我们直接使用这个镜像就好啦！

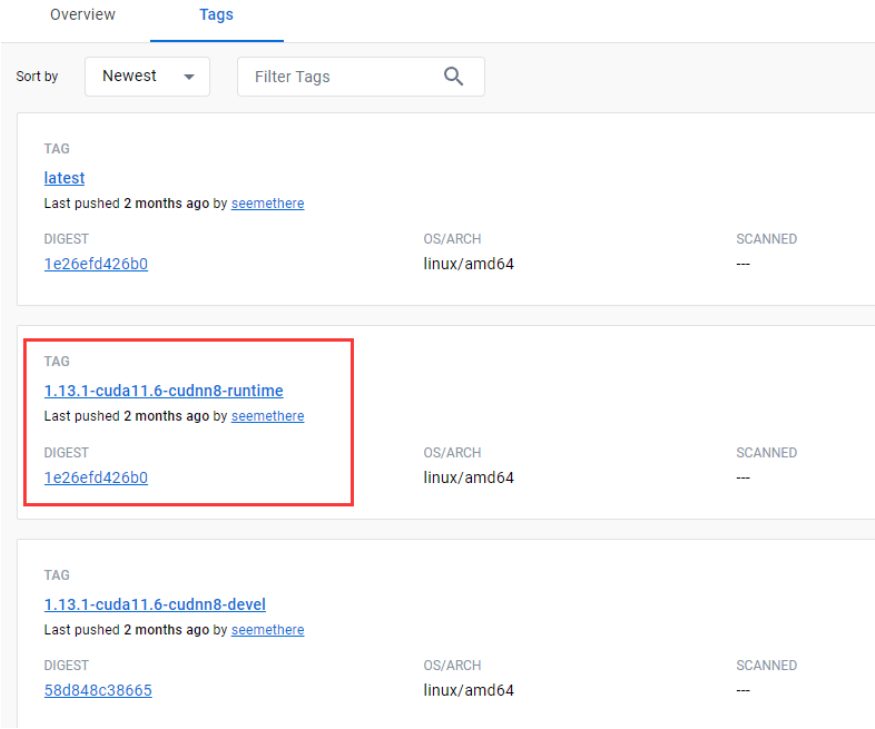


Figure 1.14: caption:pytorch-image-tags-web

下载这个镜像前，还需要登录的。首先去注册个账号，然后打开终端，输入 `dockerlogin` 登录。

然后就可以通过这条命令下载这个镜像了：

```
$ docker pull pytorch/pytorch:1.13.1-cuda11.6-cudnn8-runtime
```

这个镜像比较大，下载需要一点时间。完成后，我们再输入 `dockerimagelist` 就可以看到这个镜像了，见图 1.15。

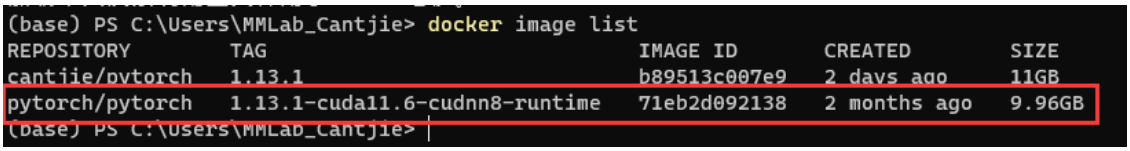


Figure 1.15: caption:docker-image-list-pytorch

1.2.3 启动容器

下载完镜像，我们该通过这个镜像启动一个容器了，我们需要到容器里看看这个容器里面是不是有我们需要的环境。

打开终端，输入

```
$ docker run -it pytorch/pytorch:1.13.1-cuda11.6-cudnn8-runtime
```

我们发现我们进入了一个 linux 系统，进去运行一下 `nvidia-smi` 试试，诶，怎么 command not found，看不到显卡。这是因为容器启动时没有给他指定 GPU。我们输入 `exit`，然后加上 GPU 参数再试一下

```
$ docker run --gpus all -it pytorch/pytorch:1.13.1-cuda11.6-cudnn8-runtime
```

进入容器后，我们输入 `nvidia-smi` 等命令，查看运行结果，如图 1.16 所示，发现正是我们所需要的环境。

1.2.4 安装新包后重新打包成镜像

但是，也有一些包在默认的镜像里是没有的，万一我们需要这些包，比如 matplotlib 包，难道我们要每次启动新的容器之后都手动通过 `condainstall` 安装一下么？不用的，我们安装一次之后，将这个容器重新打包成一个新的镜像就好了！我们之后再用，就用新的镜像了。

在刚才启动的容器里，我们输入 `condainstallmatplotlib`，安装完成后，输入 `exit` 退出容器。回到 windows 下的命令行，输入 `dockerps-a` 查看所有容器，如图 1.17 所示，我们发现刚刚安装了 matplotlib 的容器的 id 为 `23dcfbd17a23`。接下来，我们运行

```
$ # docker commit <containerID> <new-image-name>:<tags>
$ docker commit 23dcfbd17a23 new-image:1.13.1
```

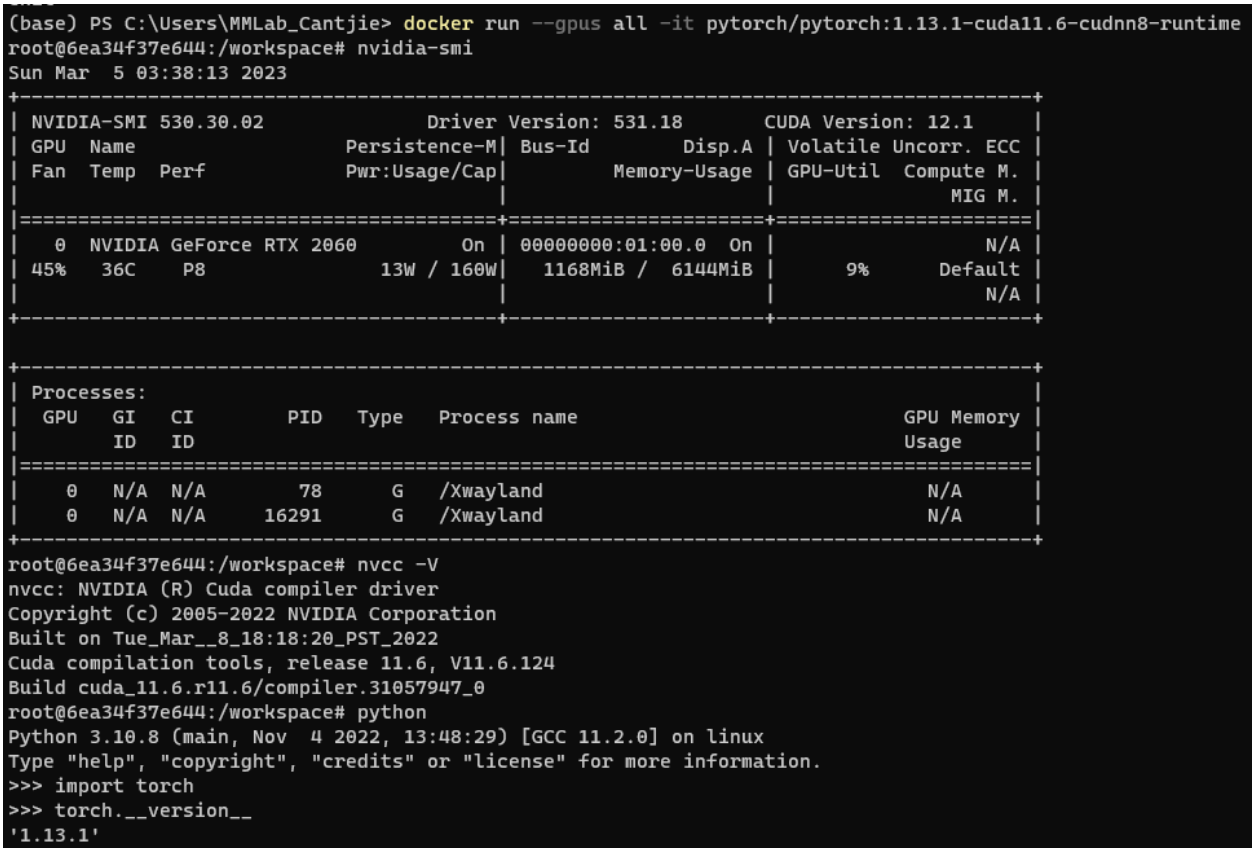


Figure 1.16: caption:docker-pytorch-container-env-check

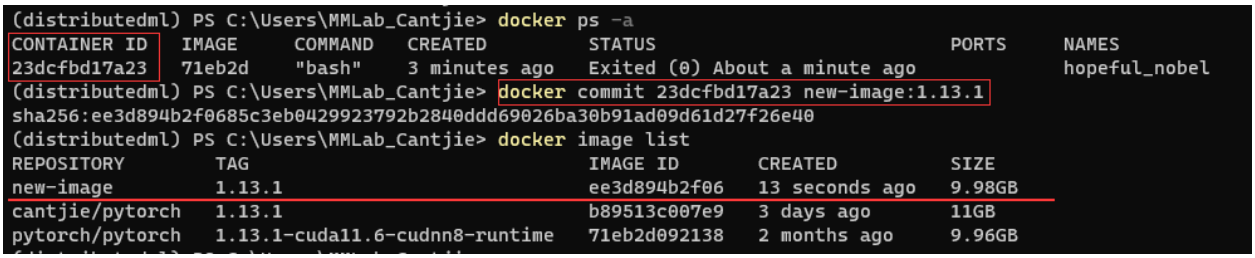


Figure 1.17: caption:docker-commit-new-image

便将容器打包成了一个镜像。输入 `dockerimagelist`，便可以看到我们新建的容器了。

以后就都可以用这个新镜像了，可是如何使用这个环境呢，我们留到完成具体实验内容的时候再来讲。

1.2.5 限制 Docker 内存占用 (可选)

由于 docker 占用内存很大，对于内存不足的电脑可能造成卡顿现象，可以通过修改配置文件限制其内存占用。

修改 C:\users\<username>\.wslconfig

```
[wsl2]
memory=6GB
swap=6GB
swapfile=E:\\wsl-swap.vhdx
```

第 2 章　实验一：梯度下降单机优化

2.1　实验内容与要点介绍

2.1.1　实验内容与要求

实验内容

- 了解常用优化算法：梯度下降、牛顿法等；
- 编写相应的算法实现代码，并进行实验；
- 分析一阶方法和二阶方法的实验结果；
- 分析确定性算法和随机性算法实验结果。

实验要求

- 在 MNIST 数据集上完成图像分类任务
- 参考 SGD 的实现示例，实现 SGDM、ADAM 两种基于梯度的优化方法，写出两个优化器类
- 绘制两种优化方法下的 loss 函数变化图像；
- 比较 SGDM 和 ADAM 两个方法的优劣，这两个方法的表现和超参数、模型选择有什么关系？请给出必要的实验和理论分析来说明；
- 使用以 tensorboard 为代表的可视化方法分析机器学习过程中的显存占用、时间开销等变化，并结合可视化结果说明训练过程中的主要时间开销或显存占用来源；

- 思考题：实现牛顿法等二阶优化方法或确定性（全批量）梯度下降，并与 SGDM、Adam 等一阶随机优化方法进行实验对比。

需要注意的是，为保证对比实验的有效性，SGDM 和 ADAM 两种优化方法应在相同的模型结构、数据处理、batch size 和训练轮数下运行；若实验在 CPU 上完成，应在结果分析中说明硬件限制带来的影响。

2.1.2 PyTorch 优化器

优化器是干什么用的

下面展示了一段简单的网络训练过程的代码，我们通过这段代码来理解 PyTorch 中优化器所发挥的作用。

```
1  def train_loop(dataloader, model, loss_fn, optimizer):
2      size = len(dataloader.dataset)
3      for batch, (X, y) in enumerate(dataloader):
4          # Compute prediction and loss
5          pred = model(X)
6          loss = loss_fn(pred, y)
7
8          # Backpropagation
9          optimizer.zero_grad()
10         loss.backward()
11         optimizer.step()
```

在这段代码中 `model` 为神经网络模型，通过 `model(X)` 调用了 `model` 中的 `forward` 方法，即进行正向传播，获得神经网络输出（第 5 行）。然后通过损失函数 `loss_fn` 计算神经网络输出 `pred` 与数据真实值或标签 `y` 的差距得到损失值 `loss`（第 6 行）。得到损失值后，通过反向传播（第 10 行），网络 `model` 中的各个参数对应的梯度将会得到更新，得到各个参数的梯度后，优化器 `optimizer` 便可以根据既定的优化算法来更新参数（第 11 行）。需要注意的是，神经网络的梯度参数并不是储存最近一次反向传播（即调用 `loss.backward()`）的结果，而是会将反向传播得到的梯度与当前储存的值相加。因此，我们需要第 9 行 `optimizer.zero_grad()` 来将神经网络 `model` 中储存的梯度值置为 0。

如果你是第一次看到类似代码，你可能还会疑惑上述代码中优化器 `optimizer` 和 `model` 似乎并没有建立联系，那为什么优化器能处理 `model` 中的参数呢？这是因为在这个函数之外，`model` 中的参数 `model.parameters()` 早就被喂给 `optimizer` 了：

```
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

如何在优化器中实现自己的算法

从上面的例子中可以看到，除了构建函数外，一个最简单的优化器只需要实现 `zero_grad` 和 `step` 方法即可。此处需要注意的有这几点：

- 当我们手动更改中参数或梯度的值时候，需要将其从计算图中分离。即在 `zero_grad` 方法中，应包含 `param.grad.detach_()`。
- 使用 Adam 算法时，由于还需要上一步优化得到的状态，因此可在初始化函数中构建一个字典用来储存状态。

2.1.3 MindSpore 优化器

除了使用 Pytorch，同学们还可以使用 MindSpore 来完成实验。

在 MindSpore 中，可以通过继承 `mindspore.nn.optim.optimizer.Optimizer` 类来自定义自己的优化器。在 Pytorch 中，我们需要实现构造函数和 `step()` 函数，类似的，在 MindSpore 中，我们需要实现构造函数和 `construct()` 函数，`construct()` 函数与 `step()` 函数作用类似。

在构造函数中，我们将神经网络参数、学习速率、衰减速率等变量存入实例。而与 Pytorch 的 `step()` 函数不同的是，`construct()` 函数需要 `gradients` 参数作为输入。并且在计算完新的神经网络参数值后，需要使用 `mindspore.ops.assign(old_param,new_param)` 函数将新的参数值赋予神经网络。

一个简单的优化器实现如下：

```
    from mindspore.nn.optim.optimizer import Optimizer
    from mindspore import ops

    class GdOptimizer(Optimizer):
        def __init__(self, params, lr=0.001):
            super(GdOptimizer, self).__init__(lr, params)


        def construct(self, gradients):
            success = None
            for param, grad in zip(self.parameters, gradients):
                update = param - self.learning_rate * grad
```

```
                success = ops.assign(param, update)
            return success
```

更多资料还可以参考 mindspore 官方文档：https://mindspore.cn/tutorials/zh-CN/r2.0.0-alpha/advanced/modules/optimizer.html?highlight=%E8%87%AA%E5%AE%9A%E4%B9%89%E4%BC%98%E5%8C%96%E5%99%A8

2.1.4 背景知识简单回顾

设待优化参数为 w，目标函数为 $f(w)$，学习率为 α, 则更新参数分为四步：

1. 计算 t 时刻目标函数对于当前参数的梯度：$g_t = \nabla f(w_t)$
2. 计算 t 时刻一阶动量 m_t 和二阶动量 V_t
3. 计算 t 时刻下降梯度：$\eta_t = \alpha \times m_t / \sqrt{V_t}$
4. 更新 t+1 时刻参数：$w_{t+1} = w_t - \eta_t$

不同优化器实质上只是定义了不同的一阶动量和二阶动量公式, 本实验涉及到的优化器有：

随机梯度下降 Stochastic gradient descent：

$$m_t = g_t \qquad V_t = 1 \tag{2.1}$$

SGDM (SGD with Momentum):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \qquad V_t = 1 \tag{2.2}$$

Adam：用修正后的 $\hat{m}_t$ 和 $\hat{V}_t$ 计算 η_t

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \qquad \hat{m}_t = m_t / (1 - {\beta_1}^t) \tag{2.3}$$

$$V_t = \beta_2 V_{t-1} + (1 - \beta_2)(g_t)^2 \qquad \hat{V}_t = V_t / (1 - {\beta_2}^t) \tag{2.4}$$

2.2 使用可视化工具观察训练期间显存与时间占用情况

当我们在进行复杂的模型训练时，常常会面临一些困惑：为什么损失下降缓慢？为什么 GPU 利用率不高？要回答这些问题，仅靠打印日志是远远不够的。我们需要更强大的剖析工具，从宏观趋势到微观瓶颈进行全面洞察。可视化工具可以为机器学习开发者提供直观的训练过程描述。TensorBoard 是用于可视化机器学习训练过程的工具，能宏观地展示损失、准确率等指标的变化趋势。pytorch profiler 可以深度剖析模型在硬件上运行时性能，能微观地定位计算、内存或通信上的瓶颈。在分布式机器学习的系统优化领域，需要对上述指标进行精确分析来优化模型训练过程。以下简要介绍上述工具的使用方法：

1. 在 conda 环境中安装 tensorboard 和 torch-tb-profiler。假设你已经配置好了 pytorch 运行环境，需要运行如下命令安装 tensorboard 以及 PyTorch Profiler 的 TensorBoard 插件 torch-tb-profiler:

pip install tensorboard -i https://pypi.tuna.tsinghua.edu.cn/simple

pip install torch-tb-profiler –upgrade

2. 在代码文件中配置 pytorch profiler，并指定存储路径:

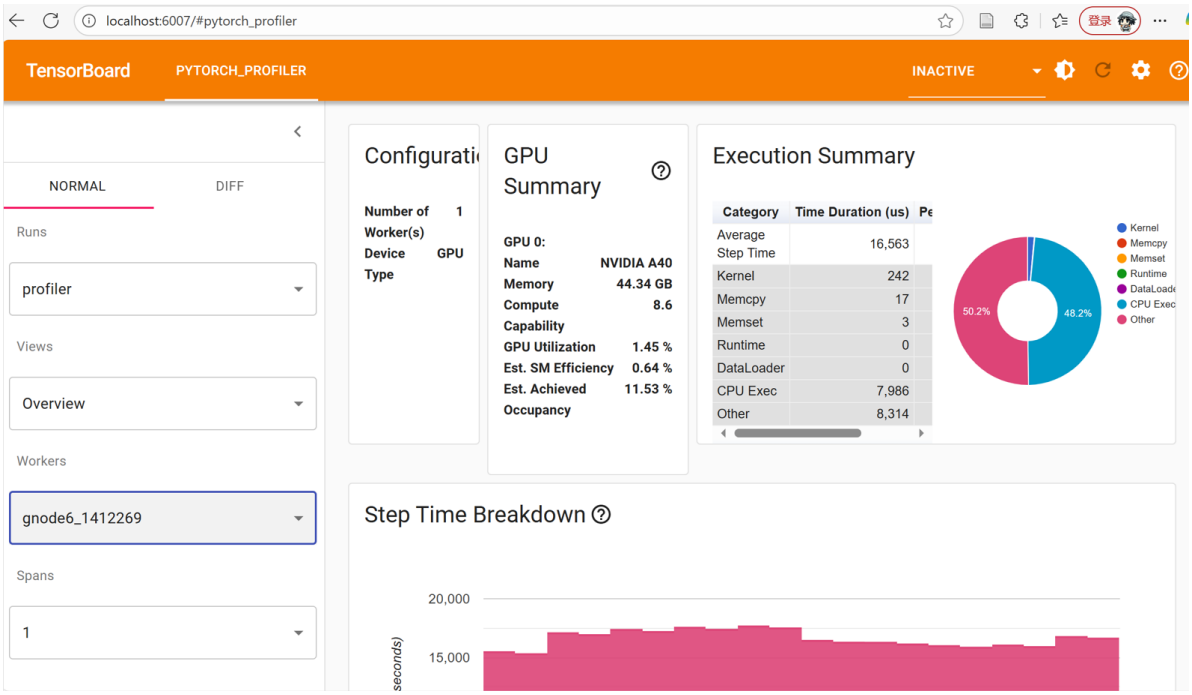
```
profiler_config = {
    'activities': [torch.profiler.ProfilerActivity.CPU,
                   torch.profiler.ProfilerActivity.CUDA],
    'schedule': torch.profiler.schedule(wait=1, warmup=1, active=10, repeat=3),
    'on_trace_ready': torch.profiler.tensorboard_trace_handler('./log/profiler'),
    'record_shapes': True,
    'profile_memory': True,
    'with_stack': True}
```

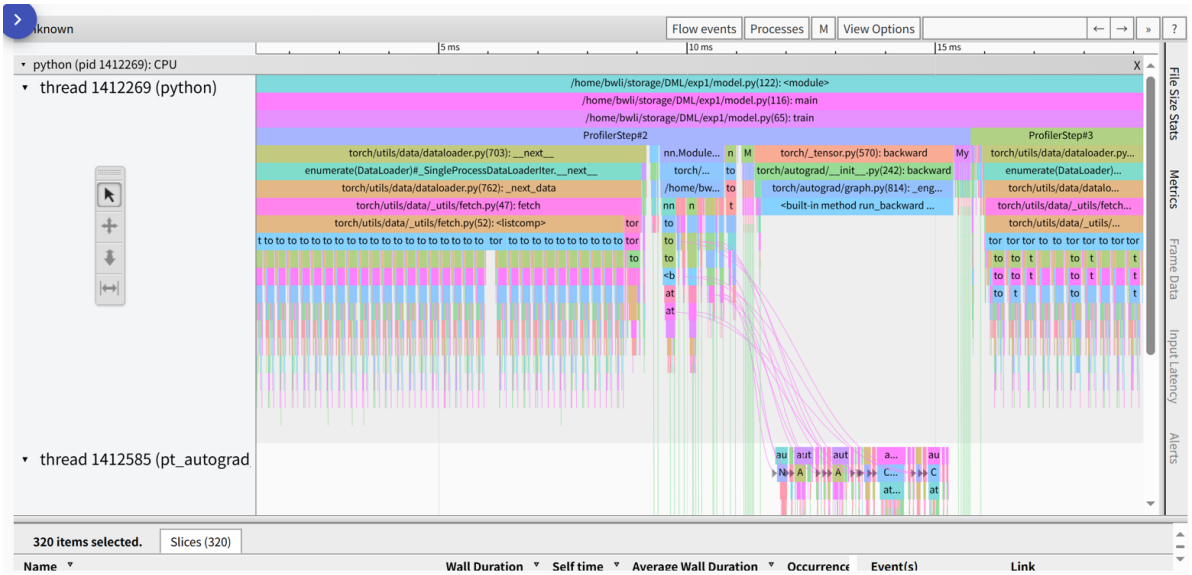
3. 训练完成后，在命令行启动 tensorboard 服务器：

tensorboard –logdir=./log/profiler

4. 在网页中访问相应的地址：

5. 在 Views 选项卡选择 Trace，可以查看数据加载、计算、反向传播的时间分布：


localhost:6007/#pytorch_profiler
TensorBoard
PYTORCH_PROFILER
INACTIVE
NORMAL
DIFF
Runs
profiler
Views
Overview
Workers
gnode6_1412269
Spans
1
Configurati
Number of Worker(s) 1
Device Type GPU
GPU Summary
GPU 0:
Name NVIDIA A40
Memory 44.34 GB
Compute Capability 8.6
GPU Utilization 1.45 %
Est. SM Efficiency 0.64 %
Est. Achieved Occupancy 11.53 %
Execution Summary
Category
Time Duration (us)
Average Step Time 16,563
Kernel 242
Memcpy 17
Memset 3
Runtime 0
DataLoader 0
CPU Exec 7,986
Other 8,314
50.2%
48.2%
Kernel
Memcpy
Memset
Runtime
DataLoade
CPU Exec
Other
Step Time Breakdown
20,000
15,000
seconds)


Flow events
Processes
View Options
5 ms
10 ms
15 ms
python (pid 1412269): CPU
thread 1412269 (python)
thread 1412585 (pt_autograd
/home/bwli/storage/DML/exp1/model.py(122): <module>
/home/bwli/storage/DML/exp1/model.py(116): main
/home/bwli/storage/DML/exp1/model.py(65): train
ProfilerStep#2
ProfilerStep#3
File Size Stats
Metrics
Frame Data
Input Latency
Alerts
320 items selected.
Slices (320)
Name
Wall Duration
Self time
Average Wall Duration
Occurrence
Event(s)
Link

2.3 使用 VSCode 与本地环境调试运行

如果你已经完成了本地环境配置 (§1.1)，那就可以打开 VSCode 进行下面的操作了：

1. 安装 Python 插件，如图 2.1 所示。

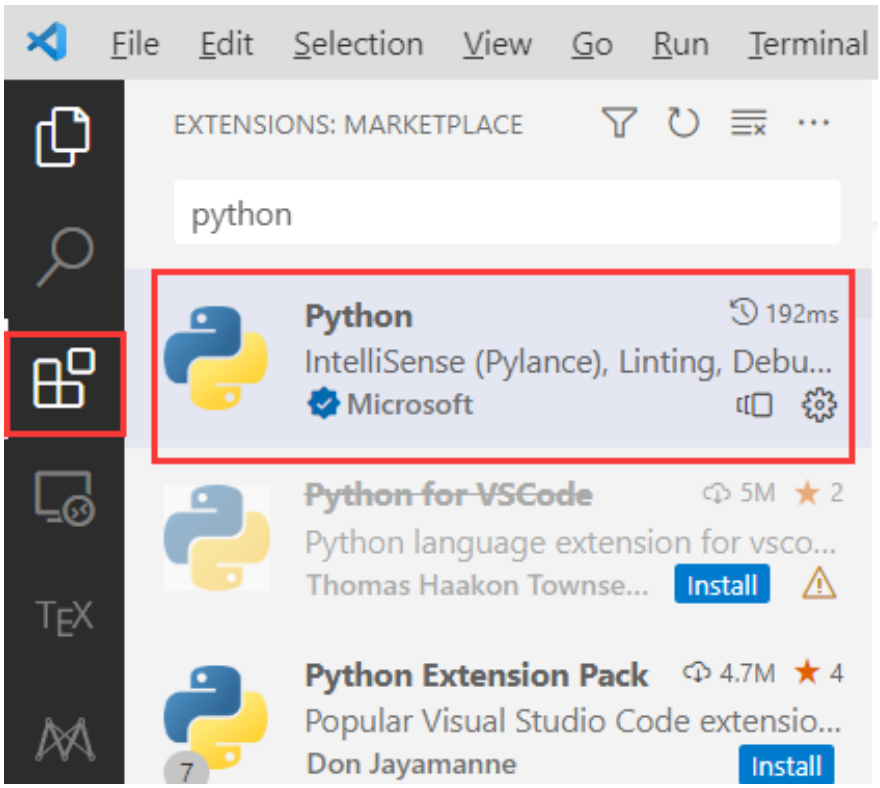


Figure 2.1: caption:task1-vscode-extension-install-python

2. 选择 Python 解释器，按下 `F1` 或 `Ctrl` + `Shift` + `P`，输入"select interpreter" 并选择 “Python: Select Interpreter” 项（图 2.2）。然后选择：select at work space level。最后选择你在 §1.1.3 一小节中创建的环境对应的解释器（图 2.3 中为助教自己创建的 distributedml 环境）。

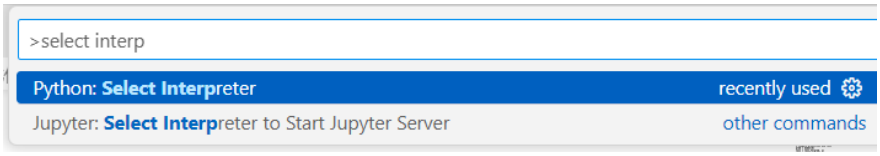


Figure 2.2: caption:task1-vscode-local-select-interpreter

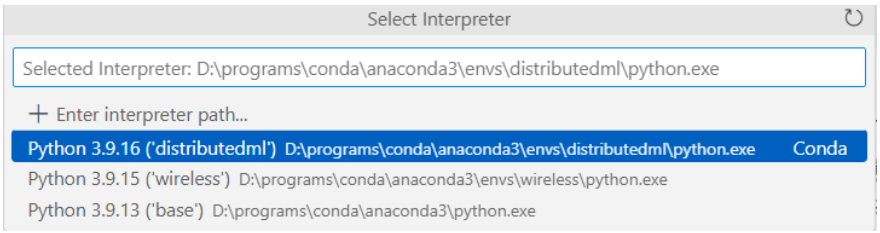


Figure 2.3: caption:task1-vscode-local-select-my-env

3. 最后，打开自己的.py 文件，可以在编辑器右上角看到一个播放形状的三角，点击它或在下拉列表中选择运行或调试，即可开始运行或调试啦。

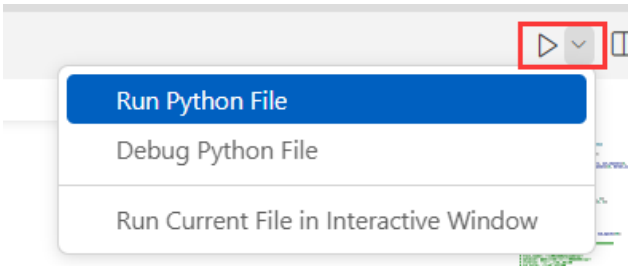


Figure 2.4: caption:task1-vscode-local-run-or-debug

2.4 使用 VSCode 与本地容器调试运行

2.4.1 启动容器并挂载本地文件夹

在 §1.2.4 一小节中，我们创建了自己的镜像，现在，我们需要先启动这个镜像（对于助教而言是 `cantjie/pytorch:1.13.1`）。但是，目前镜像里面可没有我们写好的代码，而且，就算我们在镜像里面写好代码，该怎么拿出来交作业呢?

为了解决这个问题，我们就需要将本地的目录挂载到容器上，在启动容器时，我们使用 `-v<host-dir>:<container-dir>` 参数，参考下面命令执行:

```
1    $ docker run -it --gpus all -v $pwd/relative/path/to/code:/workspace
   cantjie/pytorch:1.13.1
```

现在进入容器后，我们可以看到，如图 2.5 所示，本地的代码已经被挂载到了 `workspace` 文件夹下。

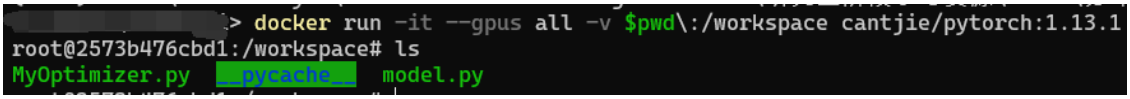


Figure 2.5: caption:task1-docker-run-with-mount

2.4.2 在 VSCode 中使用容器

首先安装 Dev Container 插件，然后按下 `Ctrl` + `Shift` + `P`，并找到 Attatch to Running Container 命令，如图 2.6。

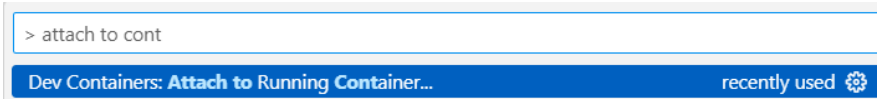


Figure 2.6: caption:task1-vscode-attach-to-container-quick-search

接下来会弹出一个新窗口，在这个新窗口中，就像在本地环境下调试运行一样在容器里调试运行即可。余下的步骤基本参考上一小节 §2.3 中的操作即可。即

- 在 VSCode 侧边栏 Explorer 栏目中打开 `/workspace` 目录。
- 在 VSCode 安装 Python 插件。
- 选择编译器为 `/opt/conda/bin/python`

2.5 使用 VSCode 与远程服务器调试运行

2.5.1 使用远程服务器计算资源

在远程服务器上创建了开发环境后，使用 ssh 链接远程环境。

首先在 VSCode 中安装 Remote SSH 插件，然后按下 `Ctrl` + `Shift` + `P`，搜索 Remote-SSH: Open SSH Configuration File 命令（图 2.7）。

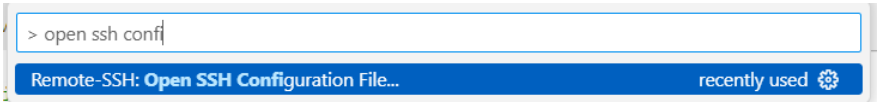


Figure 2.7: caption:task1-open-ssh-config-file

在下拉列表中选择 C:\Users\<username>\.ssh.

```
2   Host raspi
3       HostName 192.168.1.201
4       User base
5
6   Host fenbu001-default-pytorch
7       HostName 10.103.9.38
8       Port 53211
9       User root
```

Figure 2.8: ssh config file demo。注意，图中示例包含两个主机。

在打开的.ssh 文件中，按照图 2.8 给出的格式，添加一个主机。其中 Host 对应昵称，HostName 为远程主机 IP。

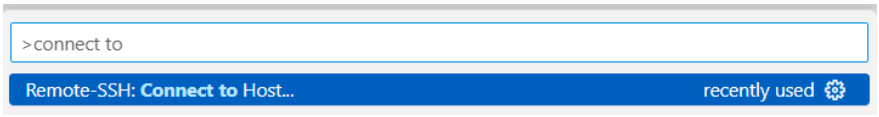


Figure 2.9: caption:task1-vscode-connect-to-host-quick-search

最后，`Ctrl` + `Shift` + `P` 并搜索 Remote-SSH: Connect to Host 命令，并在后续选择刚刚创建的主机信息。

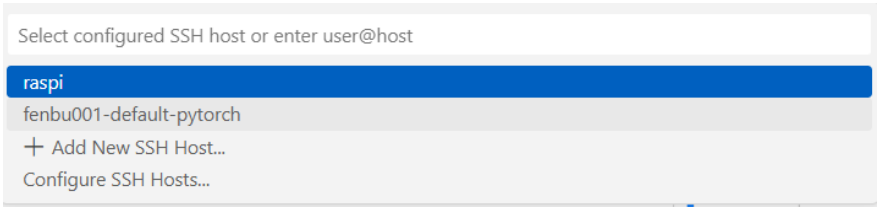


Figure 2.10: caption:taks1-vscode-connect-to-certain-host-quick-search

在弹出的窗口等待连接，并输入密码。余下的步骤又和上一小节 §2.4 一样了：打开文件夹、安装 Python 扩展、指定解释器。此处不再赘述。

2.5.2 使用华为云计算资源

手动配置

同使用远程服务器资源 §2.5.1 一样，我们也通过编辑 C:\Users\<username>\.ssh 中的 config 文件，来配置 ssh 链接。在配置前，首先进入华为云的开发环境实例中查看实例的 ssh 地址，如图 2.11 所示，该实例对应的用户名、地址和端口分别为 `ma-user`、`dev-modelarts-cnnorth4.huaweicloud.com`、`30194`。



Figure 2.11: caption:task1-huawei-modelarts-ssh-address

打开 config 文件，在原来的基础上加入如图 2.12 所示内容。注意根据你的实例和秘钥对修改其中 HostName、Port、User、IdentityFile 字段。

```
11  Host ModelArts-notebook-46f7
12  HostName dev-modelarts-cnnorth4.huaweicloud.com
13  Port 30194
14  User ma-user
15  IdentityFile C:\Users\MMLab_Cantjie\.ssh\KeyPair-d838.pem
16  StrictHostKeyChecking no
17  UserKnownHostsFile /dev/null
18  ForwardAgent yes
```

Figure 2.12: caption:task1-huawei-modelarts-ssh-config-demo

然后又和上一节一样了，用 Remote-SSH 插件的 Connect to Host 命令接入该实例即可。

使用 ModelArts 插件自动配置

聪明的你可能已经发现，在华为 ModelArts 平台中，开发环境实例最右侧操作的更多选项里有一个 VS Code 接入选项，图 2.13。如果我们已经安装了 VS Code 的 Remote-SSH 插件，我们直接点击“VS Code 接入”就可以开始远程开发了。



Figure 2.13: caption:task1-huawei-modelarts-vscode-auto-connect

第一次点击的时候，可能会提示你没有安装一个 ModelArts-HuaweiCloud 插件，如图 2.14，我们点击 Install and Open 即可。

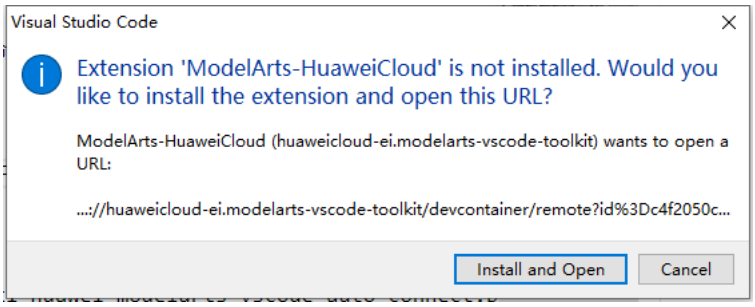


Figure 2.14: caption:task1-huawei-modelarts-vscode-auto-connect-extension-error

这时如果你打开 C:\Users\<username>\.ssh\config 文件，你会发现刚刚安装的插件的作用其实就是帮你自动写入图 2.12 中的内容。

2.5.3 使用密钥对登录远程服务器（可选）

在使用服务器计算资源和 docker 容器的时候，你可能已经发现，每次登录都要输入密码，有点麻烦。我们现在可以使用密钥对来实现免密登录远程服务器或容器。

具体而言，在本地（以助教的 windows 为例），打开终端，使用 `ssh-keygen` 命令，生成一对秘钥对：

```
ssh-keygen -t ed25519
```

然后在提示下按下三次回车，当然你也可以选择自定义这些内容，不过一般使用默认的设置和空的 passphrase 就可以了。

我们按照第一次按回车时提示的目录，找到这对密钥对，对于 windows，一般是在：C:\Users \<username>\.ssh 目录下的 `id_ed25519` 私钥文件和 `id_ed25519.pub` 的公钥文件。

接下来我们需要将公钥文件发送给服务器或容器。如果你的终端可以运行 `ssh-copy-id` 命令，那么你只需要在终端运行下面的命令即可将公钥发送。

```
ssh-copy-id user@serverip
```

然而，如果你的终端找不到 `ssh-copy-id` 命令，那么发送过程会稍微麻烦一些：手动将公钥内容写入服务器的 `~/.ssh/authorized_keys` 文件中：

```
# first ssh into the server or container
# copy the content of your <id_ed25519.pub> into <authorized_keys>
vi \~/.ssh/authorized_keys

# give the <authorized_keys> propoer permissions
chmod 600 ~/.ssh/authorized_keys
```

这样就完成公钥的发送了。之后你从终端 ssh 到服务器，就不再需要密码了。

可能的问题：权限错误

如果你在 ssh 时遇到了权限错误的提示，那很可能是你的本地的私钥的权限出问题了。找你的私钥文件，右键查看属性，在 < 安全 > 标签页点击编辑。如果你发现 < 组或用户名 > 栏目中有多个项目，如图 2.15 所示（其中马赛克挡住的应该是你目前登录的用户），那么你需要将除了你自己之外的其他用户都删掉，变成图 2.16 所示的那样。

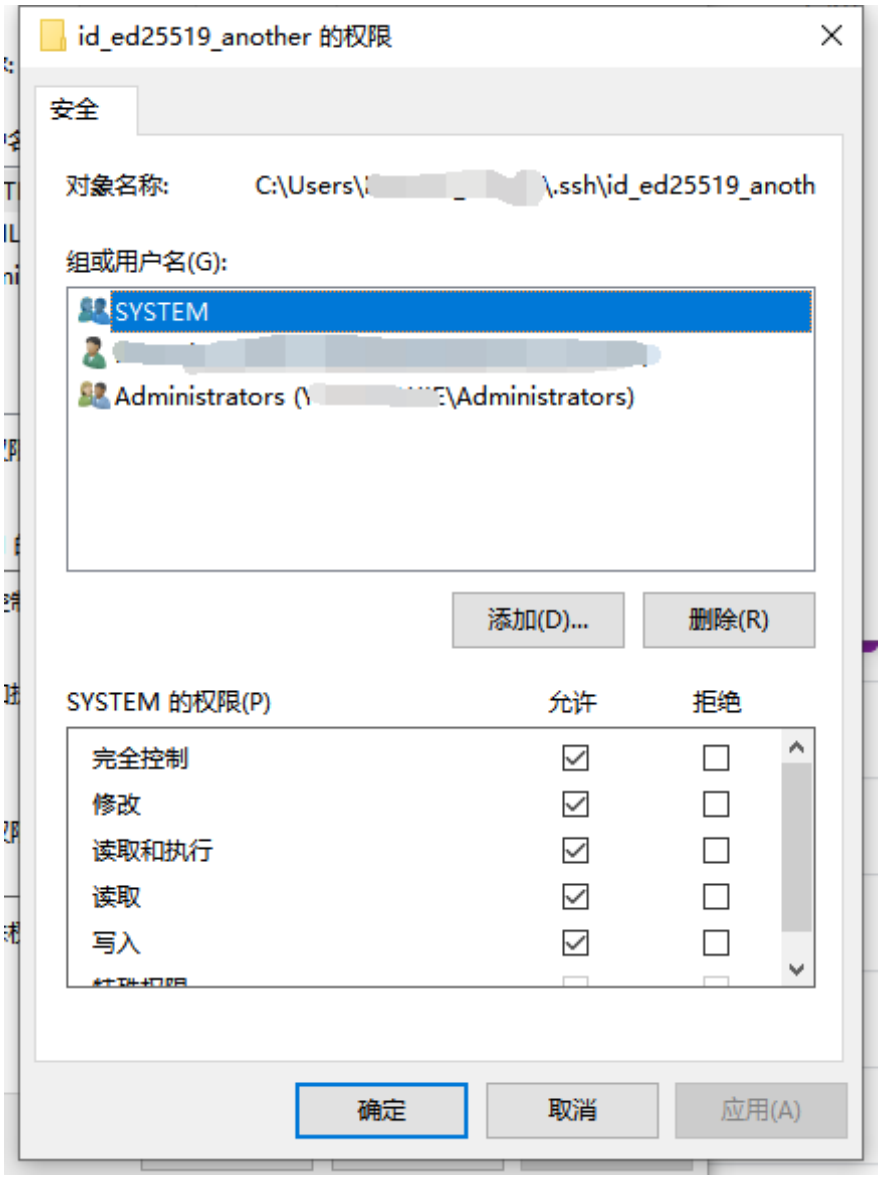


Figure 2.15: caption:task1-ssh-private-key-permission-in-windows

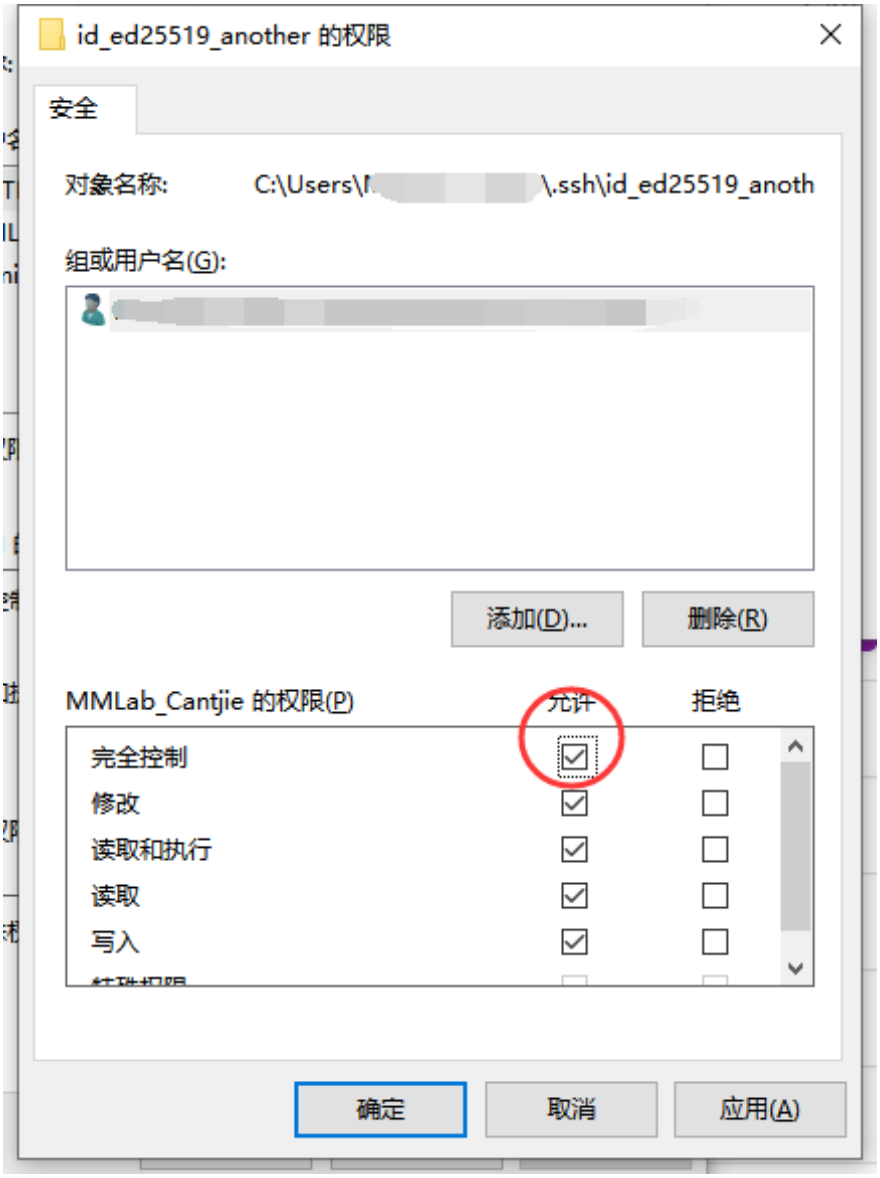


Figure 2.16: caption:task1-ssh-private-key-right-permission-in-windows

第 3 章　实验二：通信模型与参数聚合

3.1　实验内容与要点介绍

3.1.1　实验内容与要求

实验内容

- 了解集体通信中的常用消息传递接口 (Reduce，AllReduce，Gather，AllGather，Scatter 等)；
- 基于集体通信进行模型参数（梯度）聚合更新，尝试使用不同聚合方式 (Sum, Mean, etc.) 对模型的参数和梯度进行聚合；
- 分析不同聚合策略对模型性能的影响。

实验要求

- 实现集体通信下的参数（梯度）聚合，基于至少 3 种集体通信原语实现梯度平均的聚合方法，并比较它们的通信时间开销，分析不同聚合策略对模型性能的影响，报告中应交代进程数、张量大小、通信后端和运行设备等实验设置；
- 在框架下设置 ‘‘计算瓶颈节点’’，给出你能想到的所有构造计算瓶颈节点的方法，并讨论瓶颈节点对集体通信的影响；
- 讨论在不同的训练阶段（例如初始阶段、后期训练阶段），不同的聚合方式会对模型收敛速度产生的影响；

- 思考题：在存在瓶颈节点的前提下，实现异步聚合算法，并与同步聚合算法比较收敛性和时间开销；
- 思考题：了解梯度压缩（如量化、Top-K 稀疏化等）的思想，讨论其可能给通信开销和模型收敛带来的影响。

3.1.2 多节点通信

在本次实验中，我们采用 PyTorch 中的 `torch.distributed` 模块（下面将简写为 `dist`）作为多节点通信的支持工具。

初始化进程组

多节点通信的第一步，是初始化进程组。因此每个节点在训练之前，需要先调用 `dist.init_process_group()` 函数来初始化进程。这个函数会阻塞当前进程，并等待其他进程加入，阻塞持续至直到所有节点（进程）都加入了进来。

对于本次实验，我们需要关注这个函数的三个输入：`backend`，`world_size` 和 `rank`。

- `backend` 指定通信后端，即实现多节点通信的底层的通信协议，对于本次实验，当在 Linux 环境下且使用 GPU 时，一般选择 nccl，其他情况下，一般使用 gloo 即可（每种后端支持的设备类型和功能有所不同，更多内容可阅读官方文档 https://pytorch.org/docs/stable/distributed.html#backends）。
- `world_size` 为进程总数。
- `rank` 指定当前进程的优先级。启动多节点时，需要为每个进程指定 rank，一般为每个进程赋值为 0 到进程总数-1 中的整数。

但是光有这三个参数还不足以让多节点（进程）之间可以发现彼此，其实还需要告诉节点主进程的通信地址和端口。这里我们采用设置环境变量的方式告诉节点们如何找到 `rank=0` 的主节点。下面这段代码展示了 `rank=0` 的节点的初始化方式。

```
    import torch.distributed as dist

    # change it to the corresponding ip addr
    os.environ['MASTER_ADDR'] = 'localhost'
    os.environ['MASTER_PORT'] = 12355
```

```
    # initialize the process group
    dist.init_process_group(backend="nccl", rank=0, world_size=2)
```

广播模型参数

完成进程组初始化后，在神经网络模型训练开始前，需要确保所有节点上的模型是一样的，因此需要将主节点上的模型的参数广播给其他所有节点。我们使用 `dist.broadcast()` 函数来同步所有节点的参数。

下面这段代码展示了广播过程，`dist.broadcast()` 需要两个参数：

- `tensor`，为需要广播或接收的数据。当广播源为当前进程时，`tensor` 将被发送给其他节点，当广播源不是当前进程时，`tensor` 将被赋为接收到的数据。
- `src`，为广播源的 rank。

```
    if get_world_size() > 1:
        for param in model.parameters():
            dist.broadcast(tensor=param.data,src=0)
```

参数聚合

神经网络模型训练过程中，就要实现参数聚合了。该部分请同学们自行完成。

3.1.3 记录 GPU 上任务的运行时间

利用 `torch.cuda.Event.elapsed_time()` 记录，示例代码如下：

```
    start_evt = torch.cuda.Event(enable_timing=True)
    end_evt = torch.cuda.Event(enable_timing=True)
    start_evt.record()
    # the time between start_evt and end_evt will be caculated
    end_evt.record()
    torch.cuda.synchronize()
    whole_time = start_evt.elapsed_time(end_evt)
```

需要注意的是，通信时间易受设备当前负载等因素影响，同一实验应重复运行多次并取平均值；同时，如上例所示，在 GPU 上计时应在记录结束事件后进行同步，避免得到不准确的结果。

3.2 使用进程模拟多节点

为了模拟多节点通信，我们可以在同一台机器上使用不同进程或容器来实现。本节介绍多节点模拟的方法。通过进程模拟的方法在本地、在本地容器中、在华为云、在学校的计算平台都是通用的。

3.2.1 手动运行多进程

启动两个终端，分别指定不同的 rank 即可。例如：

```
    # first process:
    $ python model.py --n_devices=2 --rank=0
    # second process:
    $ python model.py --n_devices=2 --rank=1
```

3.2.2 使用 torch.multiprocessing 自动创建多进程

通过 `torch.multiprocessing` 中的 `spawn()` 函数即可让该函数自动帮我们创建多个进程，其中，我们需要关注该函数的三个参数：

- `fn` 为函数名，将作为生成的进程的入口。
- `args` 为 tuple 元组类型。每个进程将通过 `fn(i,*args)` 的方式调用，其中 `i` 即为所生成进程的 rank，从 0 开始逐次递增 1。
- `nprocs` 为生成的进程总数，即前文所指的 `worldsize` 或 `n_devices`。

下面一段代码简要说明了 `spawn()` 函数的使用方法。详情可参考 `model-mp.py`。

```
import torch.multiprocessing as mp
def main(rank, args):
    pass
if __name__ == "__main__":
    args = parse_args()
    mp.spawn(main, (args,), nprocs=args.n_devices)
```

3.3 使用容器模拟多节点

“容器就类似于虚拟机了，那通过容器模拟多节点岂不是更真实?”不知道有多少同学也和助教一样这样以为过。那我们就来尝试一下看起来更高端的容器模拟多节点吧。

注意，由于我们只在本地安装了 docker 并自定义了镜像（§1.2.4），所以下面的内容针对的是在本地使用容器模拟的过程。当然只要你掌握了方法，在远程的平台上也是一样使用的。

乍一看上去很复杂，多个容器之间的通信怎么处理呢？其实完全不用担心，我们只要使用 docker compose 就可以了，它会帮我们自动配置好同一组容器下的网络。

3.3.1 Docker compose 介绍

如果你也在 windows 下，并且安装了 docker desktop，那么 docker compose 已经是自带得了，不需要额外安装了。docker compose 就是通过使用一个模板文件（yaml 格式）来定义一组的相关联的应用容器的程序。我们首先来看一下这个所谓的模板文件是什么样的叭。

我们以助教发给大家的 `docker-compose.yml` 为例，我们取其中的一部分先简单分析一下这个文件的内容。

```
services:
    node01:
        # container_name: node01
        image: cantjie/pytorch:1.13.1
        volumes:
          - .:/workspace      # <host(local) dir (should start with . or /)>:<dir in container>
        command:              # python /workspace/model.py --n_devices=1 --rank=0 --gpu=0
          - python
          - -u
          - /workspace/model.py
          - --n_devices=2
          - --rank=0
          - --gpu=0
          - --master_addr=localhost
          - --master_port=12378
        deploy:               # make GPU accessible in container
          resources:
            reservations:
              devices:
                - driver: nvidia
                  count: 1
                  capabilities: [gpu]
```

文件中定义了两个 `services`，每个 `service` 就对应了一个容器，对于每个容器的配置，以 `node01` 为例，我们通过 `image` 指定了镜像，通过 `volumes` 指定了文件挂载路径（参考§2.4.1），通过 `command` 指定了容器启动后需要执行的命令，下面这个写法实际上是告诉容器执行这条语句：

```
$ python -u /workspace/model.py --n_devices=2 --rank=0 --gpu=0 \
    --master_addr=localhost --master_port=12378
```

其中 `-u` 表示将 Python 中 `print` 命令的输出以 unbuffer 的方式输出，这是 docker 容器的一个特性，如果不加 `-u`，我们只能在训练完成、代码跑完之后才能看到程序输出的结果啦。

最后的 `deploy` 则是让容器能够使用宿主机的 GPU（`deploy` 这一段是网上复制来哒，细问我也不懂啦）。

至于 `node02`，除了 `rank` 外，只有 `master_addr` 与 `node01` 不同，对于 `node01` 来说，master 就是自己了，而对于 `node02` 来说，master 当然是 `node01` 了。

你可能要问，那为什么这里不是写 master 的 ip，而是写“node01”就行了呢？这就是 Docker compose 的方便之处了，他会自动修改容器内的 hosts，也就是“node01”就是 `node01` 这个节点的“域名”了。

3.3.2 通过 Docker compose 启动容器

将这个文件和 `model.py` 放到同一个目录下，然后终端 `cd` 到这里，输入 `dockercomposeup` 就完成啦，我们就可以看到程序已经开始训练了！如图 3.1 所示。

这里还需要注意的是，由于我们在 yaml 中，并没有使用 `container_name` 为容器指定名字，因此 Docker 生成容器的时候，会按照 `<dir>-<service-name>-<number>` 命名方式为我们的容器命名，如果你的 `docker-compose.yml` 处在一个中文名称的文件夹下，系统很可能会报错的。放到英文命名的文件夹下就好了。

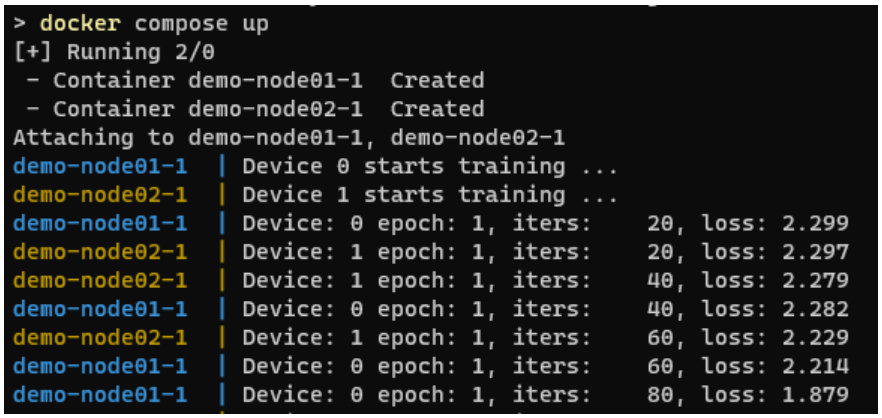


Figure 3.1: caption:task2-docker-compose-up

第 4 章　实验三 (1)：数据并行

4.1　实验内容与要点介绍

4.1.1　实验内容与要求

实验内容

- 了解常用数据划分策略；
- 编写相应的算法实现代码进行数据划分；
- 分析不同数据划分方法对模型的影响。

实验要求

- 请根据你的理解，在实验报告中完整地描述分布式训练加载数据过程中，Dataset、Sampler 、DataLoader 三类之间的关系，最好附流程图
- 在模拟多节点的 DML 系统中，实现包括随机采样和随机划分的划分方式即 2 种 Sample 类，并实现数据并行地训练模型
- 分析数据并行相对于单机训练的性能指标提升，分析不同数据划分方法对模型性能的影响
- 思考题：考虑 4 个节点并行进行训练任务，但是，节点 1 只能接触到 0，1、2，节点 2 只能接触到 3、4、5，节点三只能接触 6、7，节点四则是 8、9，有什么方法可以提升训练性能？提出你觉得可行的数据划分策略或梯度聚合策略，最好有实验或理论

推导支撑。

4.1.2 数据集、加载器和采样器

在 Pytorch 中，分布式训练加载数据过程中涉及到这样三个重要的类，即 `Dataset`、`Sampler`、`DataLoader`：

- `Dataset` 类：它直接接触源数据，将数据总数目交给 `Sampler`，将提取数据的接口交给 `Dataloader`。
- `Sampler` 类：定义 `DataLoader` 遍历数据索引的方式。
- `DataLoader` 类：在得到 `Sampler` 提供的数据索引后，去 `Dataset` 中提取数据，并将得到的数据用于训练。

他们之间的关系可以用图 4.1 来表示。

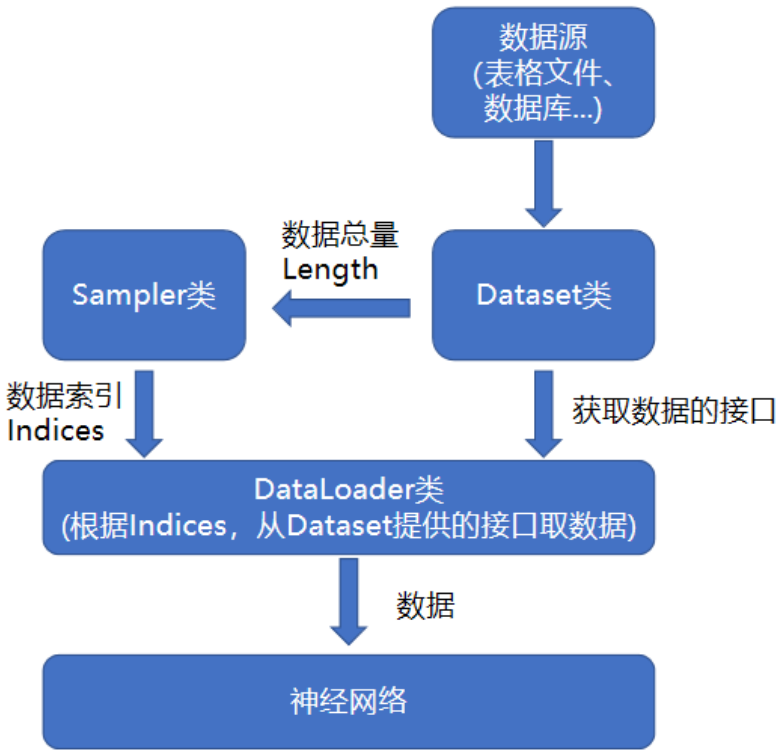


Figure 4.1: caption:task3-sampler-dataset-dataloader

在我们的实验中，我们主要需要实现 `Sampler` 类，除了构造函数外，还需要实现该类

中的 `__iter__()` 方法和 `__len__()` 方法。在 `__iter__()` 方法中，需要返回包含样本序列号的一个迭代器，譬如，一段最简单的迭代器可以这样实现：

```
def __iter__(self):
    indices=list(range(len(self.dataset)))
    return iter(indices)
```

`__len__()` 方法则应当返回上述迭代器中的数据个数。除此之外，我们也可以尝试在 `Sampler` 中实现其他功能，譬如 `set_epoch()` 方法，并在每一轮训练前调用该方法，以避免每一轮训练都得到同样的 indices 序列。

实现 Sampler 后，建议自行检查各节点实际分配到的样本索引和类别分布是否符合预期，例如打印各节点分到的索引列表或类别统计，以确保数据划分方式按照设想生效。

在本实验中，同学们可以利用实验 2 中学到的知识，尝试使用 `multiprocessing` 或自己编写 docker-compose.yaml 来模拟多节点

此外，验收时会检查每个节点实际读取的数据，而不只看程序是否运行成功；涉及随机性的实验，请在报告中记录随机种子或说明每轮的采样策略，以保证结果可复现。

第 5 章　实验三 (2)：模型并行实验

5.1　实验内容与要点介绍

5.1.1　实验内容与要求

实验内容

- 了解常用模型划分策略；
- 编写相应的算法实现代码进行模型的划分；
- 分析不同模型划分方法对模型的影响。

实验要求

- 使用 RPC 相关 API，实现模型并行训练
- 将模型拆分成两部分，分别在不同节点（进程）上进行训练
- 根据实验结果，说明两个子模型分别部署在哪些节点（进程）上，以及前向传播的数据流
- 根据实验结果，分析模型并行对分布式系统性能的影响
- 思考题：在更大模型或更深的网络中，如何选择合适的模型划分策略?

模型并行实验调试难度较高，建议先使用小模型在 CPU 环境下验证 RPC 逻辑正确后，再进行性能分析与比较；若模型并行未能带来加速，也应结合通信开销和模型规模分析原因。

5.1.2 RPC 框架介绍

在本实验中,我们将使用 Remote Procedure Call(即 RPC)框架,对应于 `torch.distributed.rpc`。这个框架中的诸多函数，将帮助我们实现模型划分的训练。但应注意，该框架尚不完全支持 CUDA，因此建议本实验在 CPU 上跑，图 5.1 为官网截图。

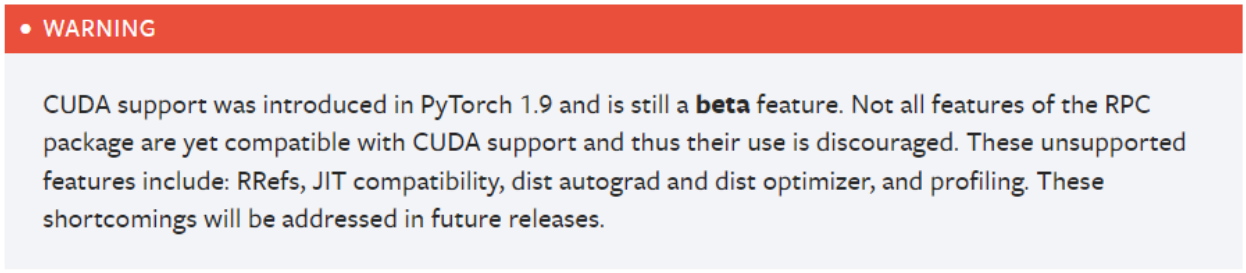


Figure 5.1: caption:task4-warning-cuda-rpc-not-compatible

神经网络定义

假设我们要把一个完整的模型拆分到两个节点上去，每个节点运行一个子模型，即 `SubModel1` 和 `SubModel2`。其工作方式为 `SubModel1` 作为第一层，`SubModel1` 的输出层再链接到 `SubModel2` 的输入层。划分时需要注意检查前后子模型的张量形状是否匹配，即前一个子模型的输出张量形状应与后一个子模型的输入张量形状一致。

为了实现这一工作方式，我们还需定义一个网络，将这两个网络组合到一起。为此，我们需要先了解 `RRef` 的概念。`RRef` 即 remote reference，它是一个远程句柄，我们可以通过这个句柄来对它所指向的内容进行操作。譬如，在主节点（进程）上，我们可以通过

```
    rref1 = rpc.remote("worker1", SubModel1, args)
```

在节点 `worker1` 上实例化 `SubModel1`，并得到这个实例化后的神经网络的 RRef。

而后,当我们需要将数据 `x` 输入到该神经网络时,我们可以在主节点上利用 `rpc_sync()` 函数调用该神经网络的 `forward` 方法:

```
    y1_rref = rref1.rpc_sync().forward(x)
```

训练过程：初始化与分布式自动求梯度

在定义过程中，我们使用了 `rpc_sync()` 和 `remote()` 函数，而为了让这些函数能顺利工作，在训练开始前，我们需要在每个节点上都完成初始化，类似于 §3.1.2 中的初始化过程，我们需要先调用 `rpc.init_rpc()` 函数。该函数有三个值得关注的输入:

- `name`，即节点名称，应保持全局唯一。
- `rank`，节点优先级。
- `world_size`，节点总数。

在完成初始化后，我们才可以实例化神经网络。在训练过程中，我们还需要使用到 RPC 框架的 `dist_autograd` 模块，该模块可以帮我们让不同节点自动完成梯度下降过程中所需的通信。下面代码是一个使用该模块的简单的例子:

```
    import torch.distributed.autograd as dist_autograd
    with dist_autograd.context() as context_id:
        pred = model.forward()
        loss = loss_func(pred, loss)
        dist_autograd.backward(context_id, loss)
```

该示例中，第二行生成了一个 `context_id`，该 `context_id` 用来指示一个节点的上下文对象，用于在反向传播中指导节点的通信。

运行

Windows 似乎不支持 RPC 框架，因此通过 docker compose 一键执行，方便极了

```
demo> docker compose up
[+] Running 3/3
 - Container demo-node03-1  Created                                    0.3s
 - Container demo-node02-1  Recreated                                  0.4s
 - Container demo-node01-1  Recreated                                  0.4s
Attaching to demo-node01-1, demo-node02-1, demo-node03-1
demo-node02-1  | Training on the worker1...
demo-node03-1  | Training on the worker2...
demo-node01-1  | Device 0 starts training ...
demo-node01-1  | Device: 0 epoch: 1, iters:    20, loss: 2.306
demo-node01-1  | Device: 0 epoch: 1, iters:    40, loss: 2.302
demo-node01-1  | Device: 0 epoch: 1, iters:    60, loss: 2.304
demo-node01-1  | Device: 0 epoch: 1, iters:    80, loss: 2.302
demo-node01-1  | Device: 0 epoch: 1, iters:   100, loss: 2.297
demo-node01-1  | Device: 0 epoch: 1, iters:   120, loss: 2.301
```

Figure 5.2: caption:task4-docker-compose-up

关于 RPC 的使用，助教总结的十分有限，还请同学们见谅。因此因此关于本章的的更

多内容，还请同学们参考教程和文档。

- 教程 Distributed RNN using Distributed Autograd and Distributed Optimizer：https://pytorch.org/tutorials/intermediate/rpc_tutorial.html#distributed-rnn-using-distri
- rpc 文档：https://pytorch.org/docs/stable/rpc.html

第 6 章　实验报告撰写要求

在撰写每项实验报告时，请注意并且避免出现以下问题：

- 结构不完整。报告一般应包含以下几个关键要素：
  1. **实验原理**: 简单交代所要求实现算法的原理，向读者交代自己的理解，让批阅人快速批阅作者的理解是否正确。
  2. **代码实现**: 通过截图或 markdown 等自带的代码块贴上关键性代码。
  3. **实验步骤，或运行参数、环境**: 交代如何实现多节点、关键参数设置为何（简要交代即可，譬如 batchsize、停止条件、训练数据集等）。
  4. **实验结果**: 通过图片、表格等形式清晰地展示结果。
  5. **结果分析或结论**
- 截图影响可读性。
  - 对代码的截图
    1. 截图后字号过大、字号过小或相邻截图字号相差过大。
    2. 截图中，文本编辑器为深色背景，连续多块截图面积宽度不一致，导致看上去像一块块膏药，阅读起来非常难受。
  - 对实验结果的截图
    1. 终端的文字性的输出结果: 如果认为有必要截图证明自己完成了实验，可以作为中间结果，但仅作为最终结果输出会严重影响可读性。结果中关于最后几轮的 loss，可以用曲线展示；结果中运行时间、准确性等可以通过表格展示。

2. 曲线展示的结果: 需要对比时，最好把两条（多条）曲线放到一张图上对比展示，而不是两张图每张图上仅一条曲线。

- 格式不清晰影响可读性。全部左对齐时，建议用 markdown 类似的格式。注意缩进、字体、字号变换、章节编号等。
- 交代不清晰。
  1. 缺乏文字性阐述。对实验原理、实验步骤、实验结果等缺乏文字性阐述。对于新引入的符号缺乏说明，对多条曲线中的图例缺乏解释等。
  2. 缺乏单位。交代运行时间时缺乏单位等。

第 7 章　实验验收列表:2026 秋季学期

本学期实验最终提交版应包含的最少内容。

7.1　实验一：单机优化算法构建

- SGDM 和 Adam 两种算法的实现代码；
- 使用可视化方法分析时间开销、显存占用等指标；
- 在 MNIST 数据集上实现图像分类任务训练过程中的损失函数变化图像。

7.2　实验二：通信模型与参数聚合

- 基于至少 3 种集体通信原语（如 AllReduce、AllGather 等）实现梯度平均的聚合方法的实现代码；
- 上述聚合方法的通信开销对比；
- 使用可视化方法分析不同聚合方式的计算通信时间差异；
- 瓶颈节点的设置与瓶颈对集体通信和性能的影响。

7.3　实验三：数据并行和模型并行

- 数据并行：随机采样与随机划分的实现代码；
- 模型并行：将模型横向划分的实现代码；

- 在两种数据并行方法下和在模型并行与否的两种情况下，比较模型性能。

第 8 章　华为云资源与 MindSpore 框架使用方法

8.1　使用华为云计算资源

8.1.1　创建开发环境

我们将使用华为 ModelArts 平台，ModelArts 是华为云提供的 AI 开发平台，同学们可在实名登陆后通过 https://www.huaweicloud.com/product/modelarts.html，点击管理控制台进入 ModelArts 管理控制台，图 8.1。



Figure 8.1: caption:huawei-cloud-modelart-homepage

进入管理控制台后，首先把区域切换到“北京四”，防止出现找不到资源的情况，图 8.2。

从左边导航栏中进入在线开发环境，即选择开发环境-Notebook。然后点击创建按钮以



Figure 8.2: caption:huawei-modelarts-beijing4

创建开发环境，图 8.3。



Figure 8.3: caption:huawei-modelarts-notebook

选择合适的镜像、资源规格、储存空间等，如图 8.4 所示。如需进行远程开发，请打开 SSH 远程开发按钮，并设置密钥对。

如没有密钥对，需创建密钥对，点击图 8.4 中的立即创建，按照图 8.5 所示步骤创建密钥对，并将私钥文件保存到本地。私钥文件，建议保存到 C:\Users\<username>\.ssh 文件夹下。

如此，我们便成功创建了一个开发环境，我们可以在此环境之下安装其他包，此处不再赘述。



Figure 8.4: caption:huawei-modelarts-environment-create



Figure 8.5: caption:huawei-modelarts-keypair-create

8.1.2 添加数据存储

当有多个开发环境实例都需要使用同一个数据集时，相比每次新建实例都从本地上传，还可以选择为每一个实例都挂载同一个数据存储空间。

点击刚才创建的实例，按照图 8.6 所示，点击添加数据存储，在弹出的窗口中编辑本地挂载目录，即新建的储存空间会挂在到该实例的什么地方。再选择并行文件系统，如此处为空，则点击新建并行文件系统，在新弹出的界面输入文件系统名称后点击立即创建即可。

在 Notebook 中挂载 OBS，不适用于对挂载文件做频繁随机修改，适用于对不同挂载大小文件对象的一次保存（上传），多次读取（下载）。



Figure 8.6: caption:huawei-modelarts-filesystem-mount

更多问题可以参考：https://support.huaweicloud.com/modelarts/index.html

8.2 MindSpore 介绍

华为开源自研 AI 框架昇思 MindSpore，是一个全场景深度学习框架，旨在实现易开发、高效执行、全场景覆盖三大目标。自动微分、并行加持，一次训练，可多场景部署。支持端边云全场景的深度学习训练推理框架，主要应用于计算机视觉、自然语言处理等 AI 领域。

8.2.1 整体介绍

华为 AI 致力于构建业界最强的 AI 算力平台，使能千行百叶的智能化转型。华为 AI 的整体框架如图 8.7 所示，其中可以看到我们在 §8.1 中介绍的华为 ModelArts 平台和 MindSpore 框架所处的位置。



Figure 8.7: caption:mindspore-whole-picture

MindSpore 框架拥有全场景 AI 计算能力，图 8.8 展示了 MindSpore 全场景 AI 计算框架架构图，其包含了 AI 计算所需的各个组件，从软件到硬件，从网络模型、API 表达层、编译优化层到底层各种计算资源硬件和其使用框架，均包含在 MindSpore 中。

MindSpore 架构具有如下特点：

- 用户态易用；

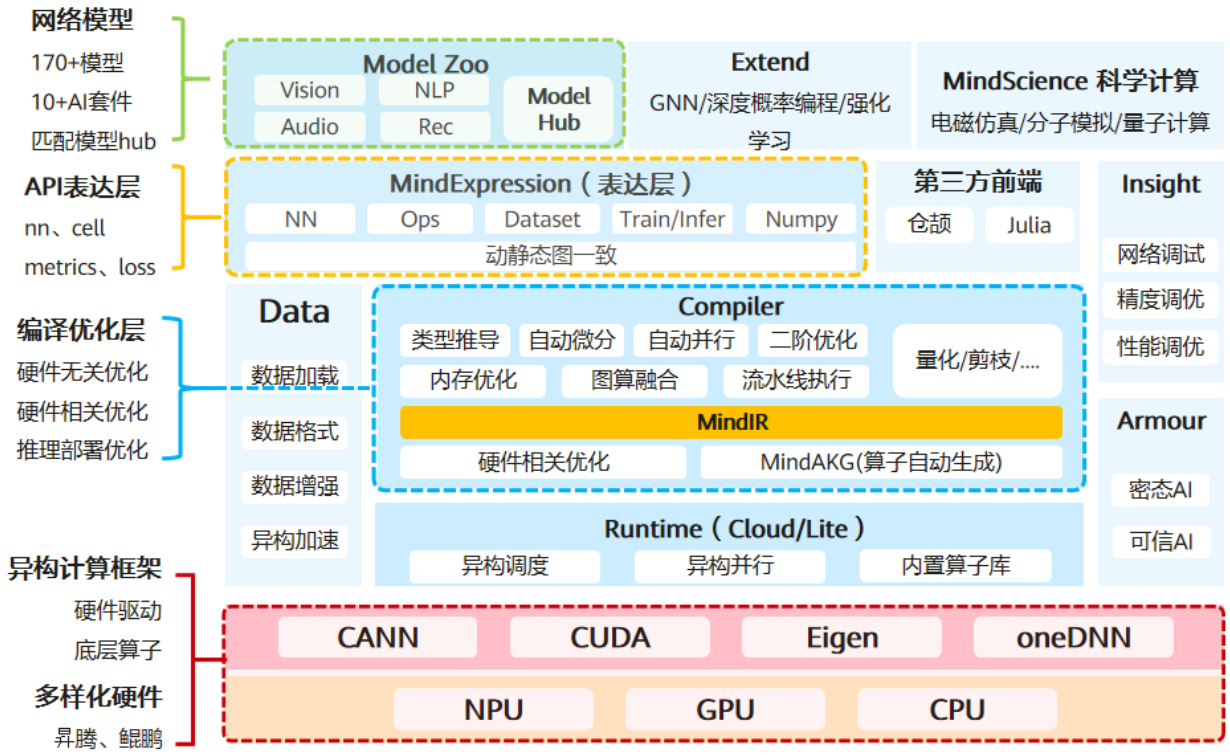


Figure 8.8: caption:mindspore-ai-architecture

- 运行态高效；
- 部署态灵活；

MindSpore 具有如下特性：

- 自动并行：通过自动并行机制、数据 pipeline 处理等手段降低超大模型训练门槛。
- AI+ 科学计算：支持 AI+ 科学计算的高阶/高纬、多范式编程。
- 通用计算 +DSA：通过图算融合对性能进行优化，自动算子生成技术简化异构（DSA）编程，发挥多样性算力的性能。
- 端边云统一的可信架构：解决企业级部署和可信的挑战。

MindSpore 包含了 MindExpress、MindCompiler、MindSpore Runtime、MindData 等多个子系统，在实验指导书中不再赘述，有兴趣了解的同学可以参考助教分享的演示文稿。

8.2.2 MindSpore 安装

安装过程也与 Pytorch 的安装类似，具体而言，打开 https://www.mindspore.cn/install，选择合适的版本、平台等后，按照网站的指示安装即可。



Figure 8.9: caption:mindspore-install

8.2.3 社区资源

- 昇腾开发者社区：https://hiascend.com
- 昇腾论坛：https://www.hiascend.com/forum/
- MindSpore 开源社区：https://www.mindspore.cn/
- ModelArts 社区：https://bbs.huaweicloud.com/forum/forum-718-1.html
- Gitee 仓库：https://gitee.com/mindspore